



COMP 4300

Intro to Game Programming

Lecture 14

Pathfinding

Steering Behavior

Movement in Games

- Movement is arguably the most important part of a large number of video games
- It allows the exploration of the game world and interaction with entities
- Not only does the player move in a game world, but the NPCs / entities as well
- Good movement makes a good game

Movement in Games

- Main video game movement problems:
 - Path-finding (how to get from A to B)
 - Path-following (how to move along path)
 - Physics (how environment reacts)
 - Animation (how entities look)

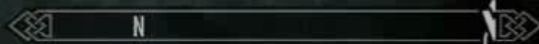
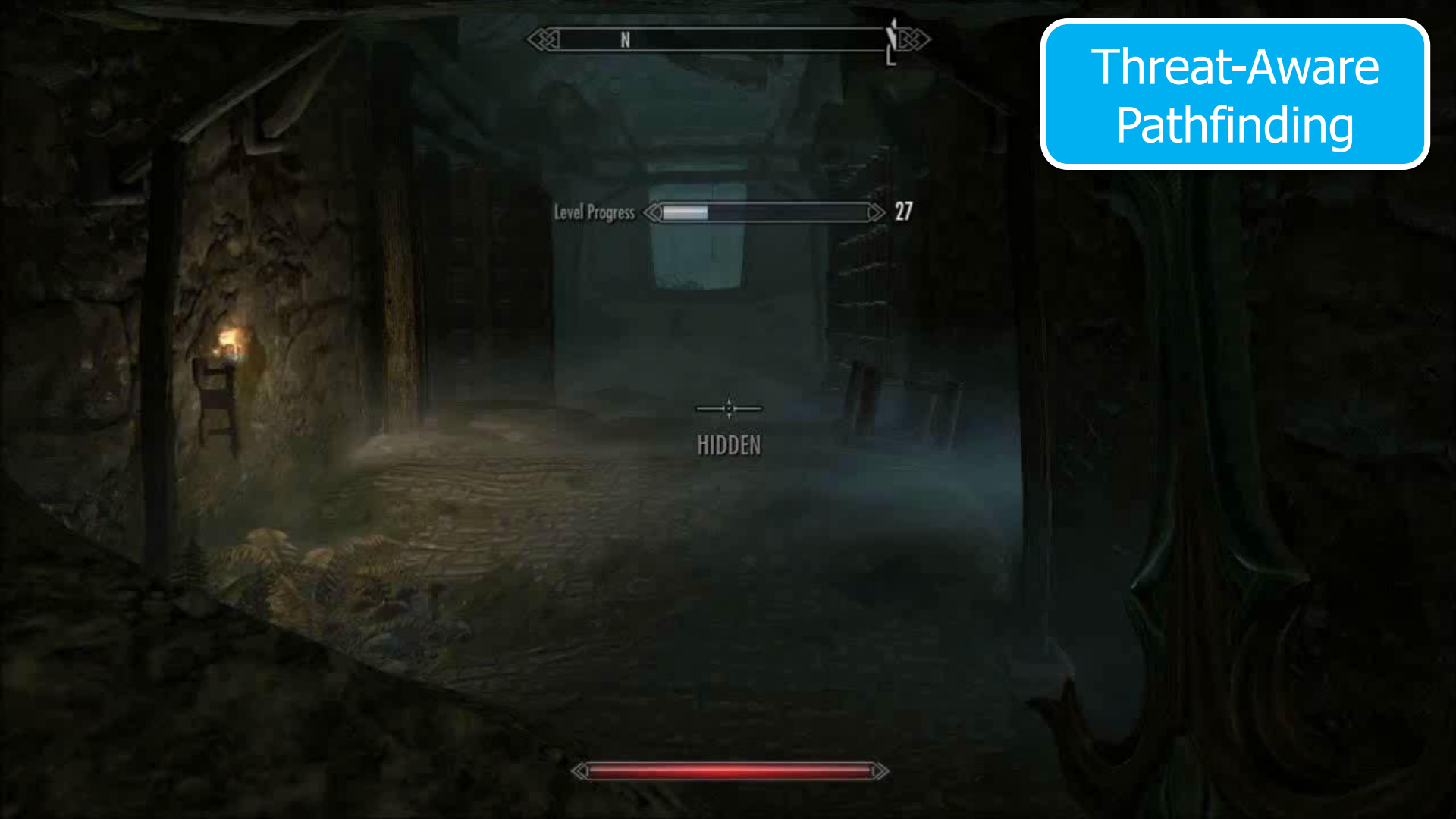
Movement in Games

- Main video game movement problems:
 - Path-finding (how to get from A to B)
 - Path-following (how to move along path)
 - Physics (how environment reacts)
 - Animation (how entities look)
- In this lecture we will deal with decision making for movement (path find / follow)

'3D' Pathfinding



Threat-Aware Pathfinding



Level Progress  27

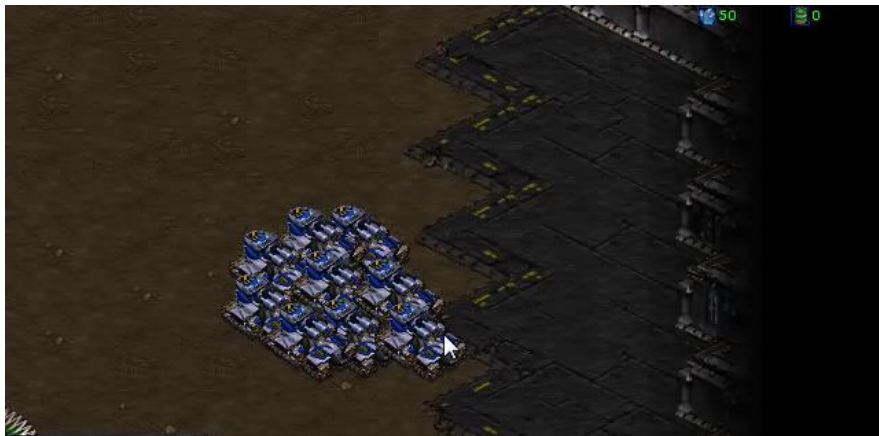


HIDDEN



Multi-Agent Pathfinding

Move as Group



Surround Enemy

Menu (F10)

Message Log (F11)

Help (F12)

Units



Went under ground!



Realistic
Behavior

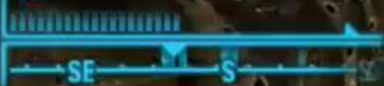
ACQUIRED ELFROOT



Realistic Paths



HP



AP



CND

5/123

Animation Matters



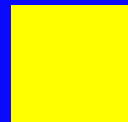
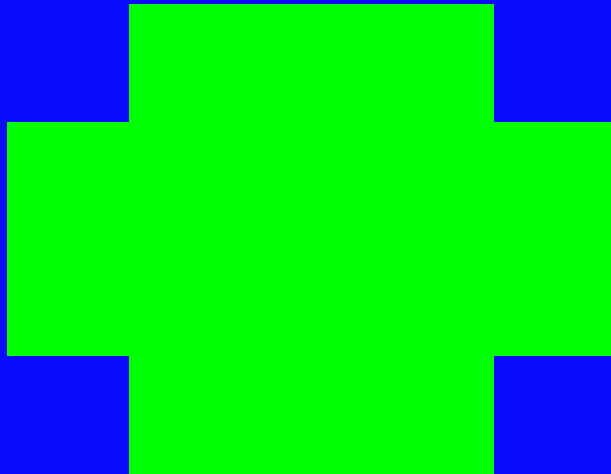
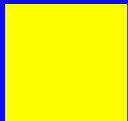
Pathfinding

- Video game maps are often filled with **obstacles** with which NPCs will **collide**
- Pathfinding is the method we use to determine which path an NPC should take from its **current** location to a **goal** location
- Pathfinding attempts to **minimize a 'cost'** of the path. It could be total length, fuel, damage taken, or an arbitrary function

Pathfinding Considerations

- Single Agent / Multi-Agent
- Single Goal / Multi-Goal
- Static (precomputed) / Dynamic (online)
- Space representation
 - Open / Grid / Graph / Polygons
- Movement directions
 - Fixed movement (Up, Down, Left, Right)
 - Arbitrary angles / movement
 - Move between nodes on a graph

Pathfinding



S

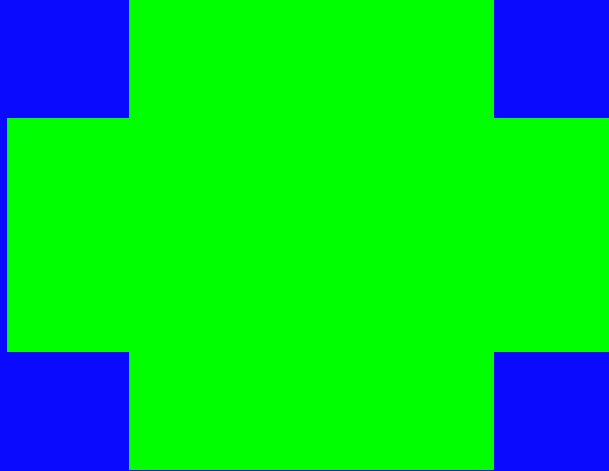
Single Agent
Single Goal

G

S_1

S_2

S_3



Multi Agent
Single Goal

G

S

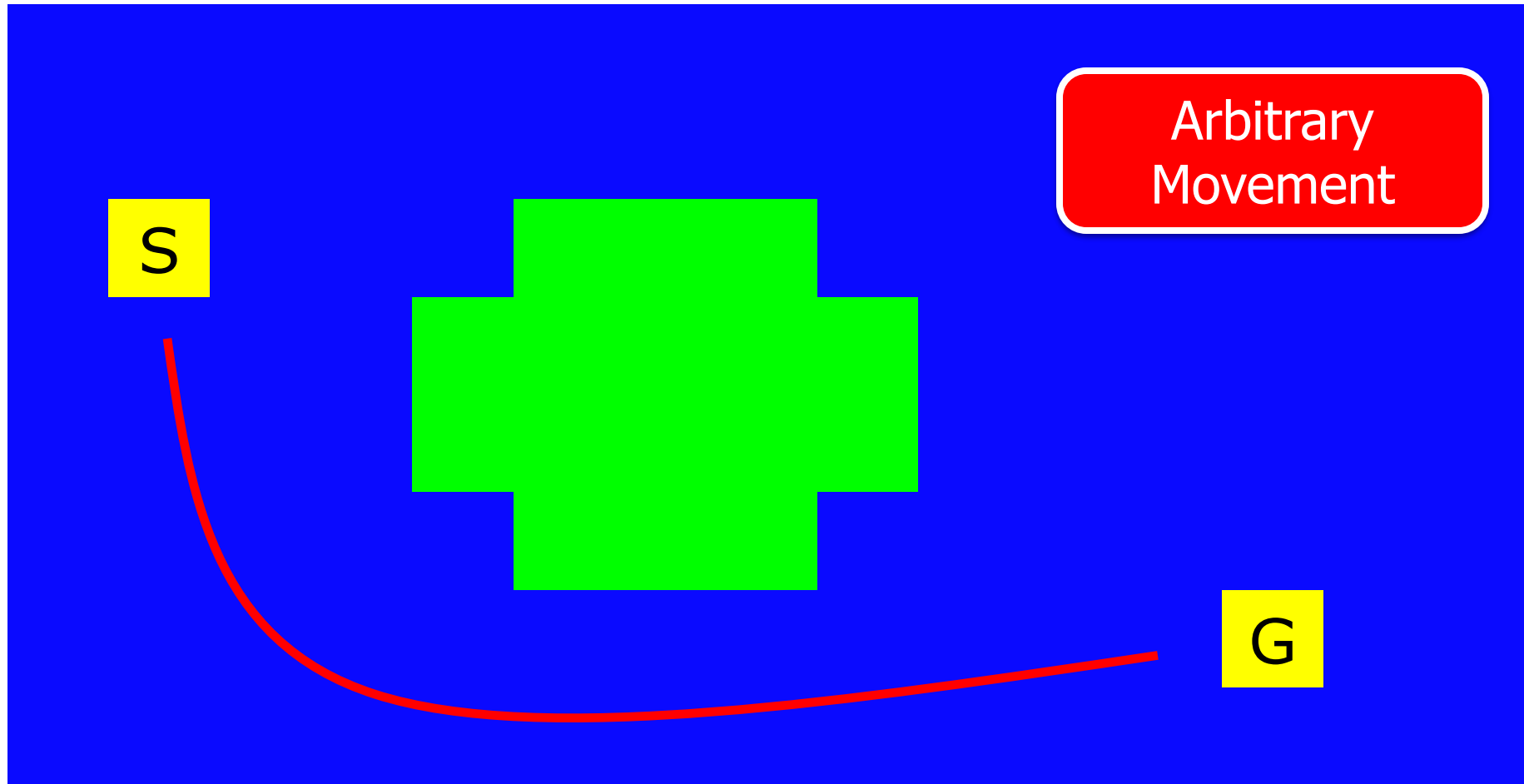
Single Agent
Multi-Goal

G_2

G_1

Space / Action Representation

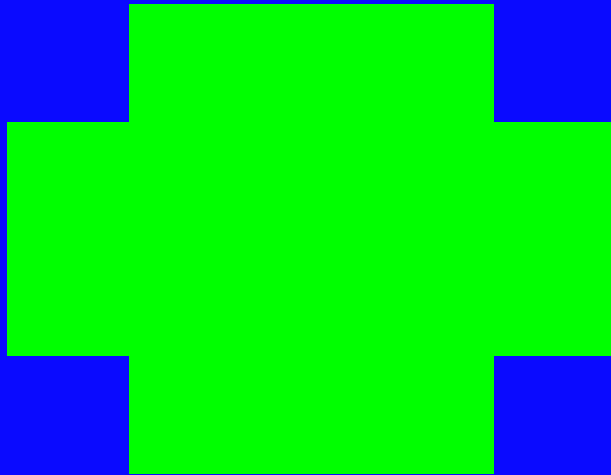
- Space and action representations are important decisions, ideas are linked
- Which representation to choose is based on many factors from the game's **design**
- The choice of space representation in the pathfinding engine will determine which movement **actions** can be performed

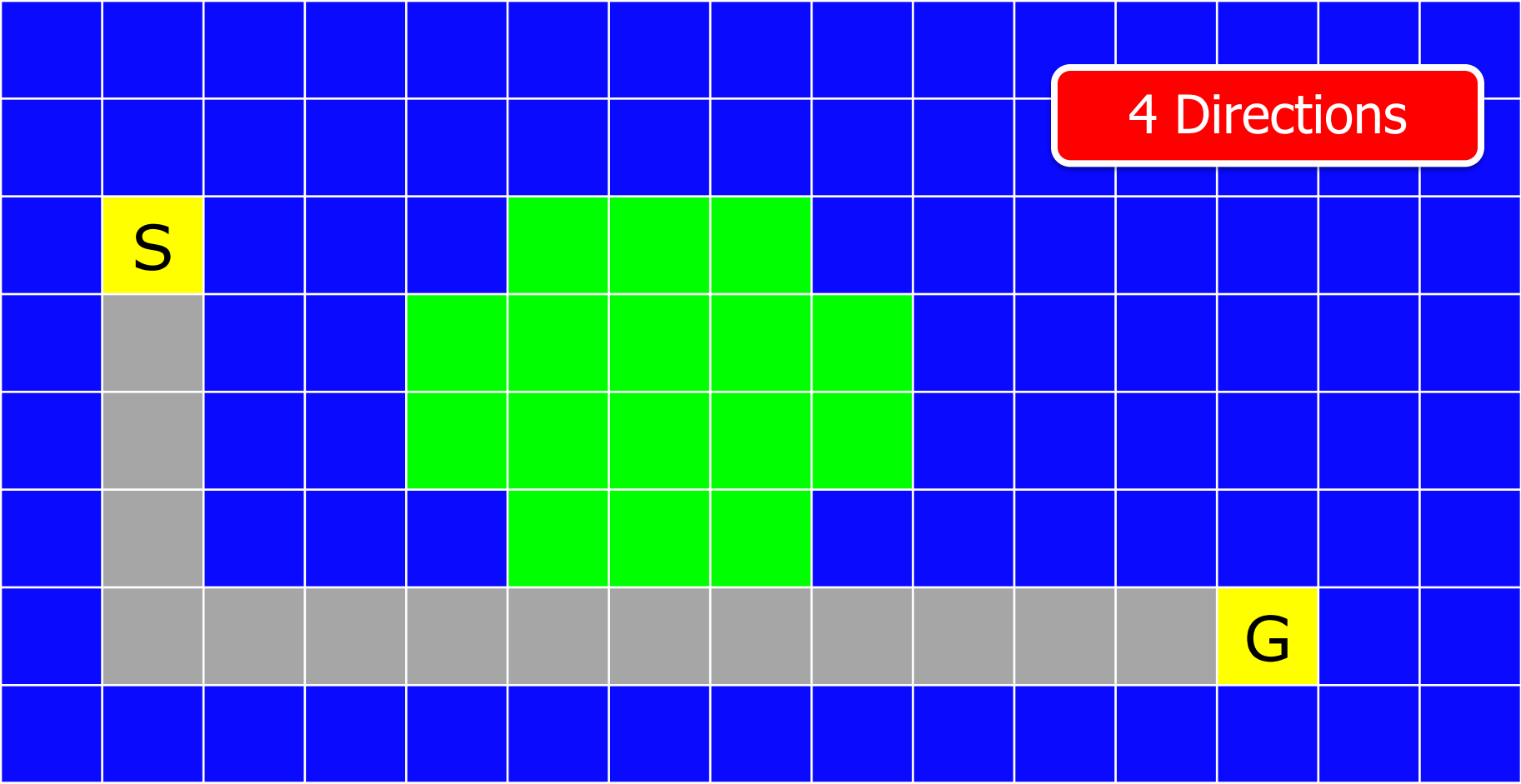


Straight Line
Movement

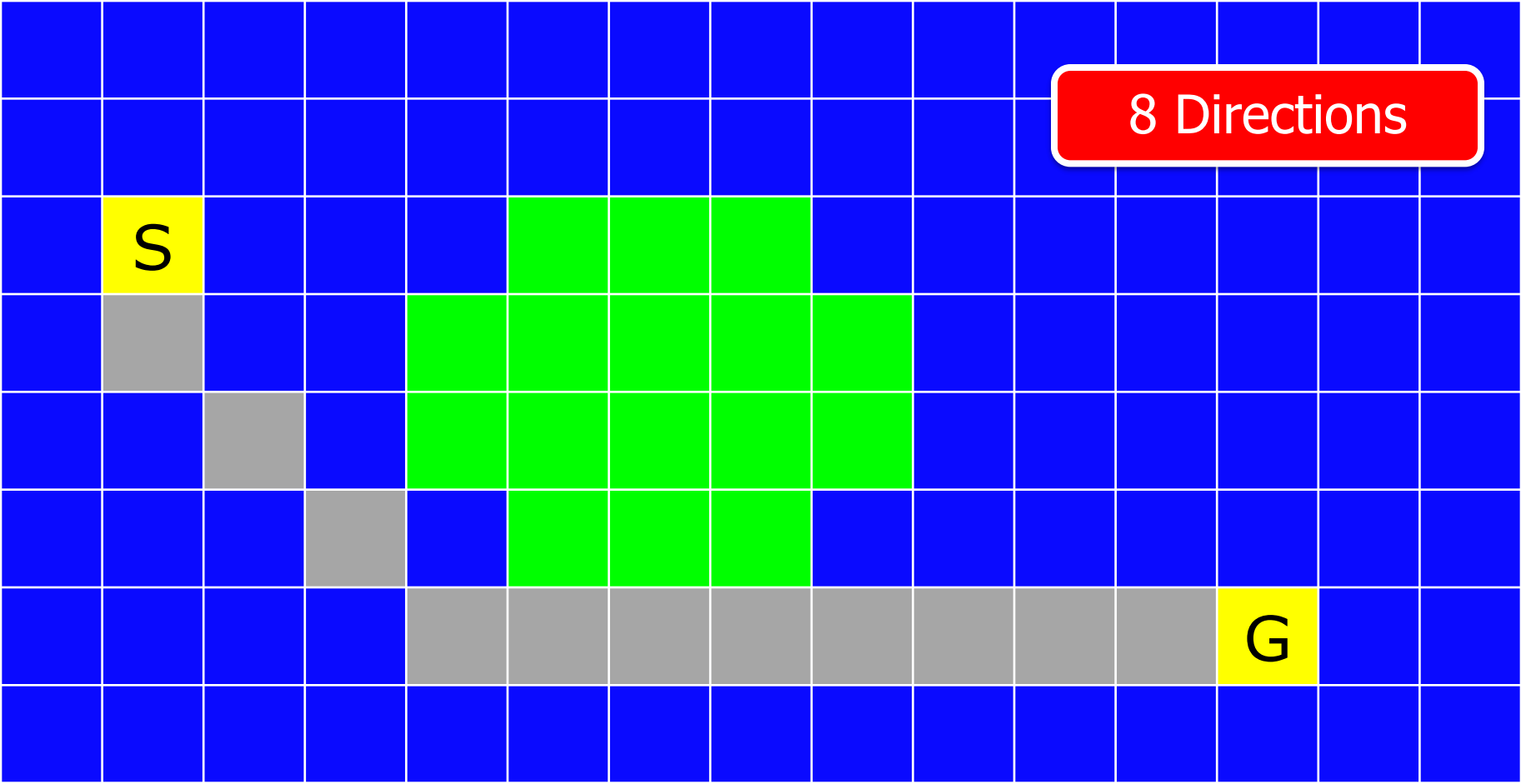
S

G

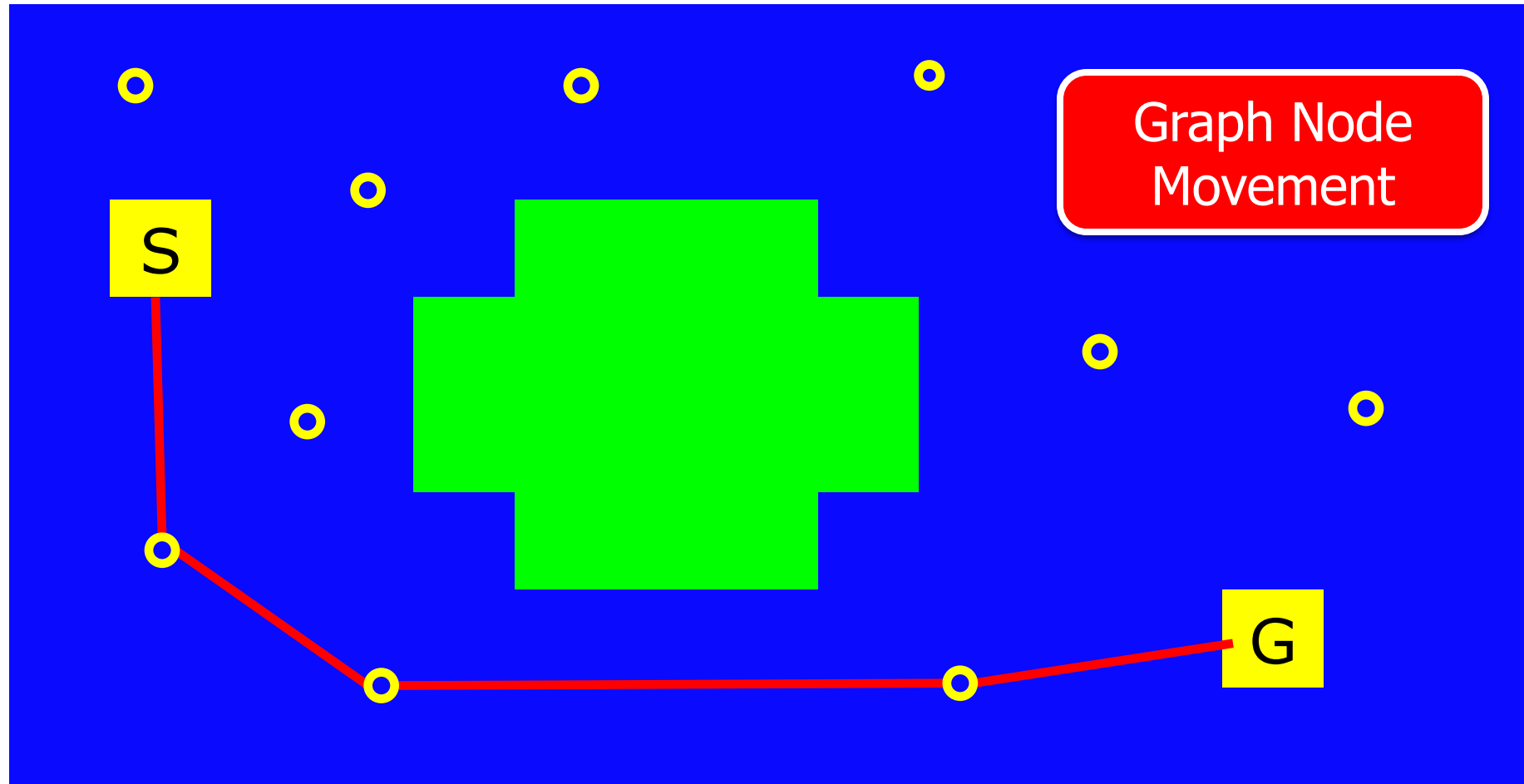


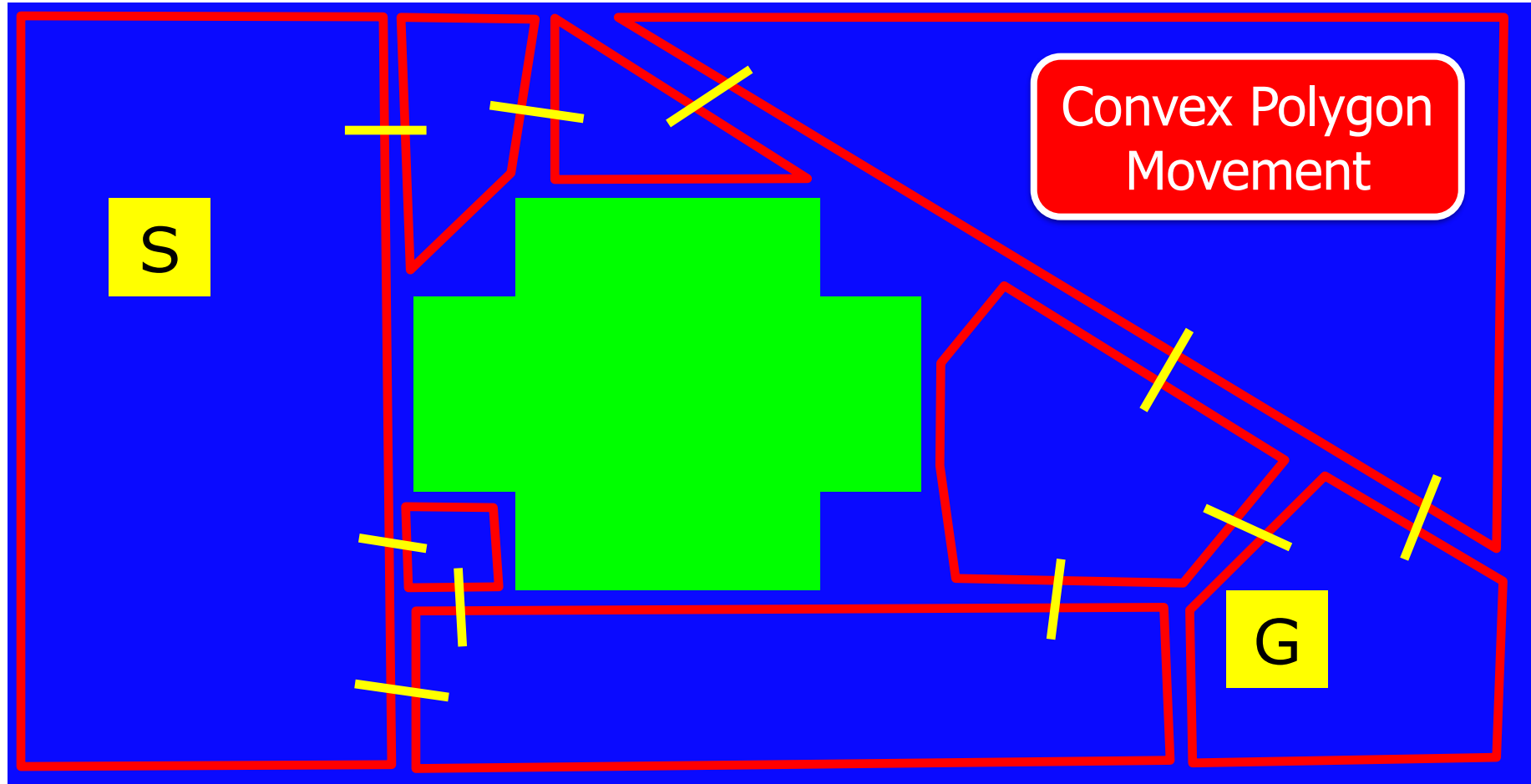


4 Directions



8 Directions



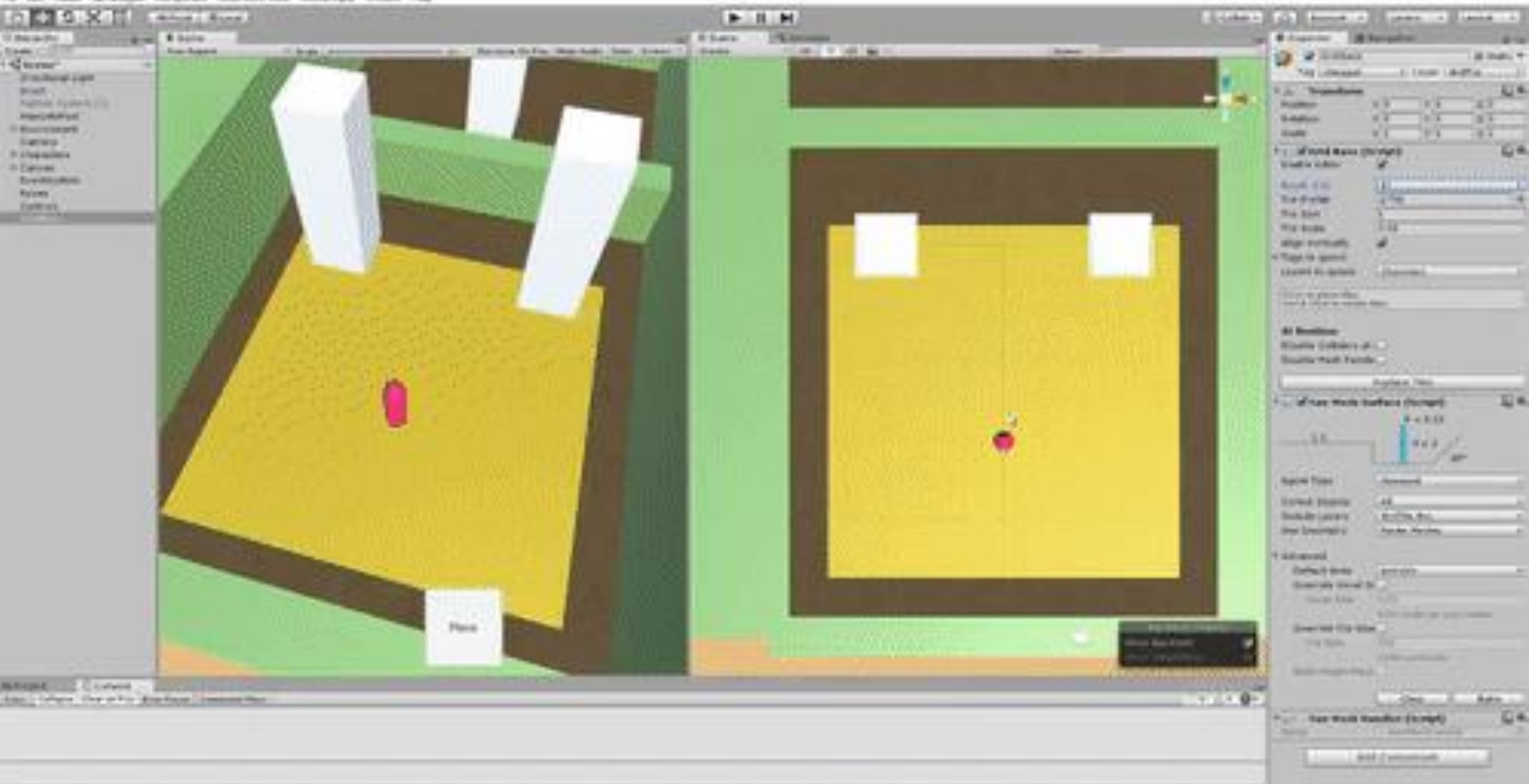


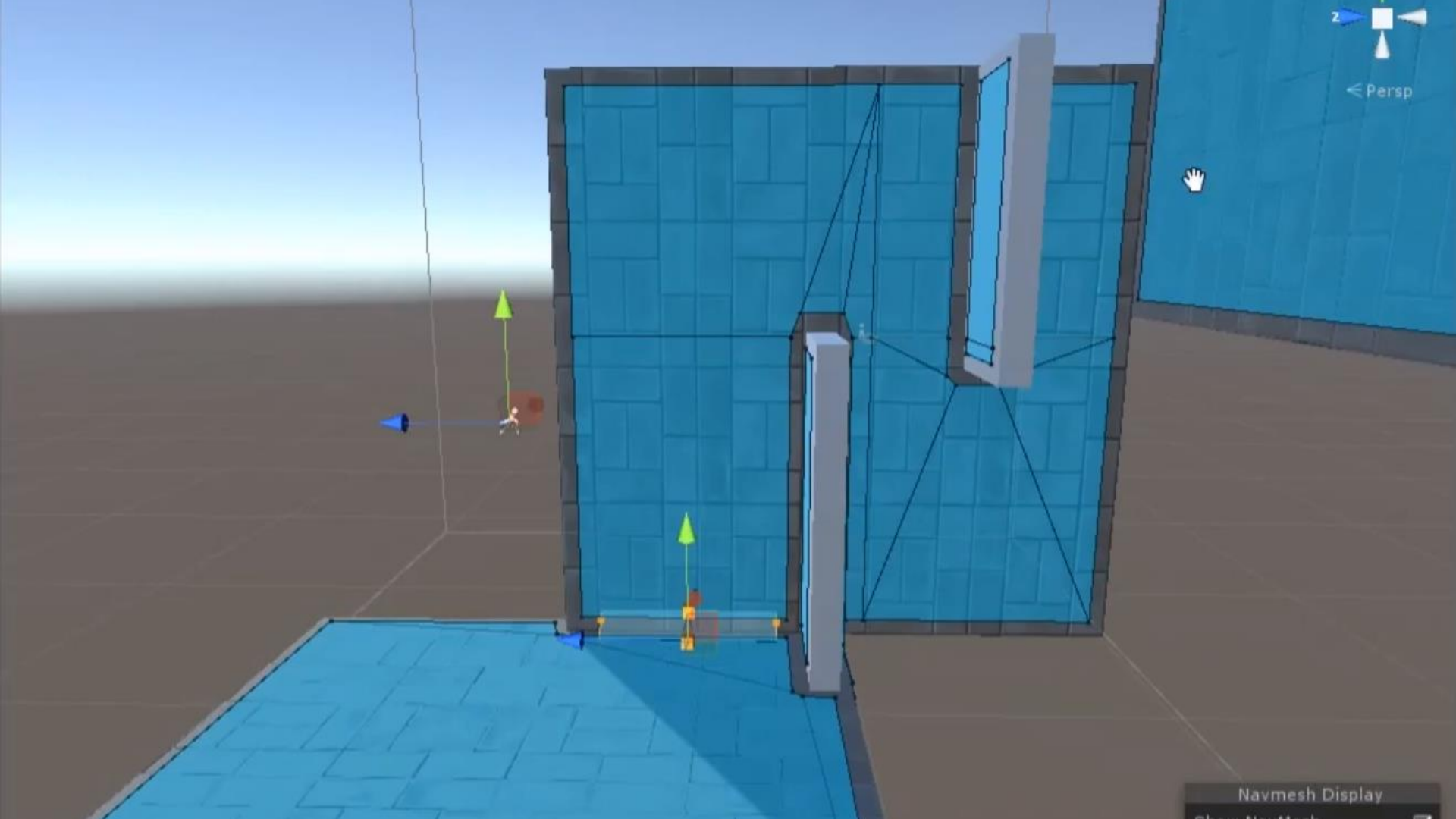
Choosing State Representation

- Choosing a state representation matters
 - **Memory size** dependent on representation
 - Defines which **actions** are possible
 - Defines which **algorithms** are applicable
- Try to find a **balance** between the computation required to find a path vs the final path accuracy
- Typically in games we choose a **sparse** state representation for easier computation, then later **refine** the path to look more realistic

Navigation Mesh

- What if my game has **arbitrary geometry**?
- Need a **graph** structure within our environment to enable pathfinding
- This graph is called a NavMesh
- Most game engines come with software to build and edit a level's NavMesh

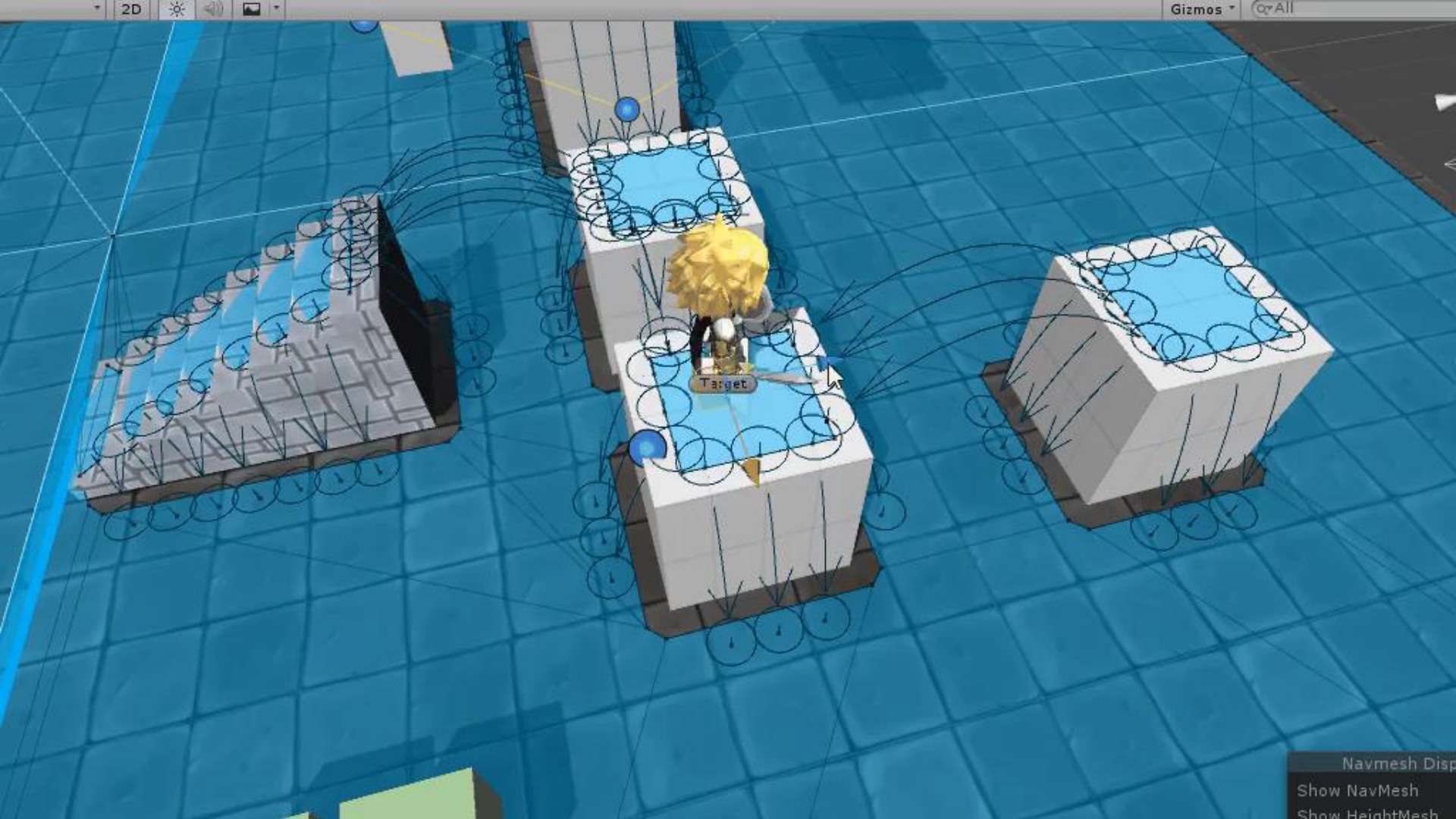




2
Persp

Navmesh Display

Show NavMesh



Single-Agent Pathfinding

- Many graph search algorithms exist for finding shortest paths, each with their own properties
- Common algorithms:
 - Breadth-First Search, Dijkstra, A*
- Most video game pathfinding has traditionally used some variant of the A* search algorithm

A* Search Algorithm

- Most well-known Best-First search algorithm
- Evaluates nodes by using $g(n)$ and $h(n)$
 - $g(n)$ = path cost so far to current node n
 - Heuristic function $f(n) = g(n) + h(n)$
 - $h(n)$ = estimate of cheapest solution via n
- Select node from open list with $\min f(n)$
- Performance depends on properties of the heuristic function

A* Search Algorithm

1. Function **AStar**(problem, h)
2. closed = {}
3. open = PriorityQueue(Node(problem.initial_state), f=g+h)
4. **while** (true)
5. **if** (open.empty) **return** fail
6. node = pop_min_f(open)
7. **if** (node.state is goal) **return** solution
8. **if** (node.state in closed) **continue**
9. closed.add(node.state)
10. **for** c in Expand(node, problem) // for each neighboring node
11. open.add(c)

Admissible Heuristic

- **Never overestimates** distance to goal
- Can be seen as 'optimistic' guesses
- $h(n) \leq h^*(n)$
- $f(n) = g(n) + h(n)$ never overestimates true cost of a path through n when $h(n)$ is admissible

Heuristic Function

Heuristic Function

Admissible?

Heuristic Function

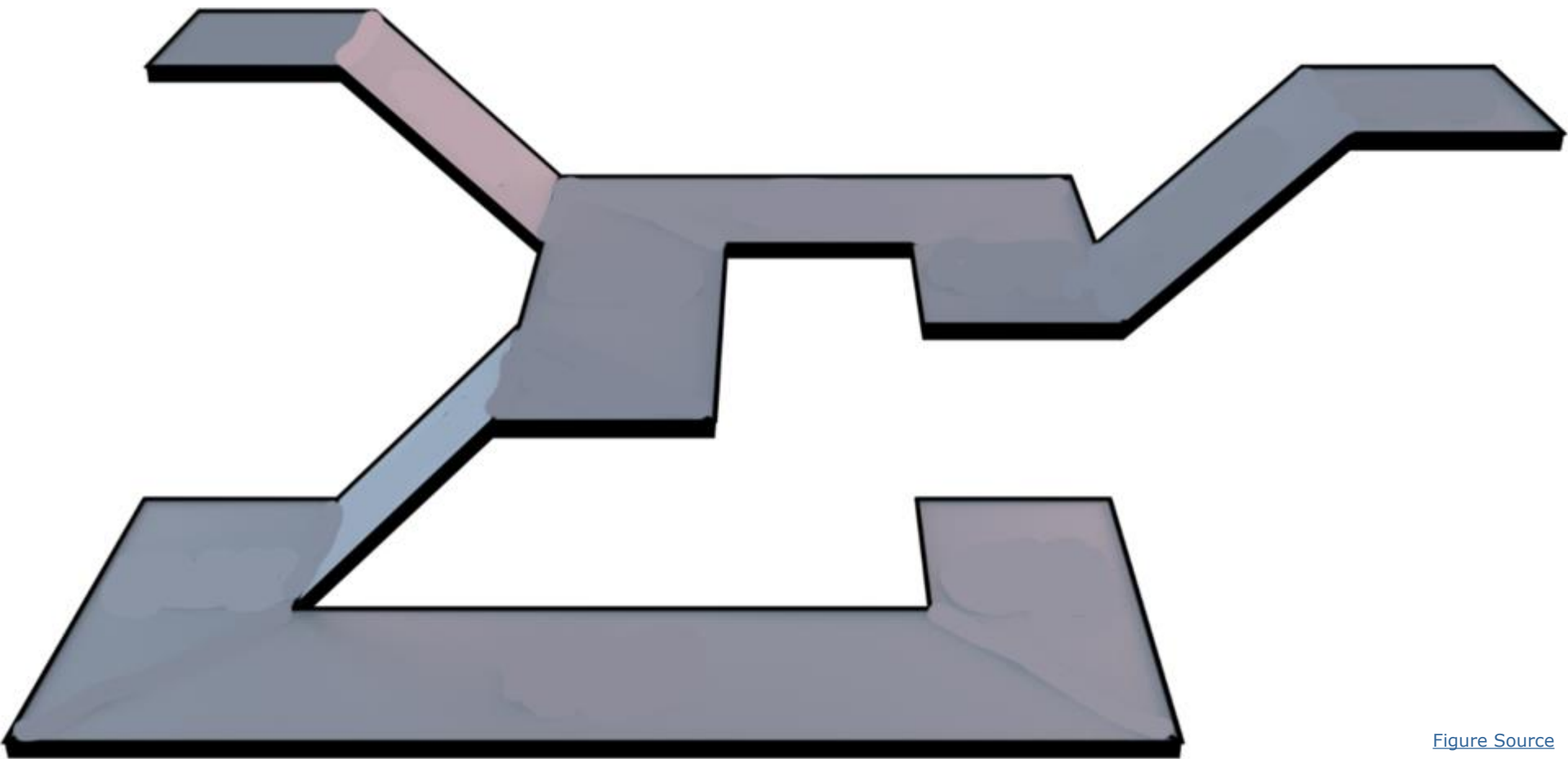
Admissible?

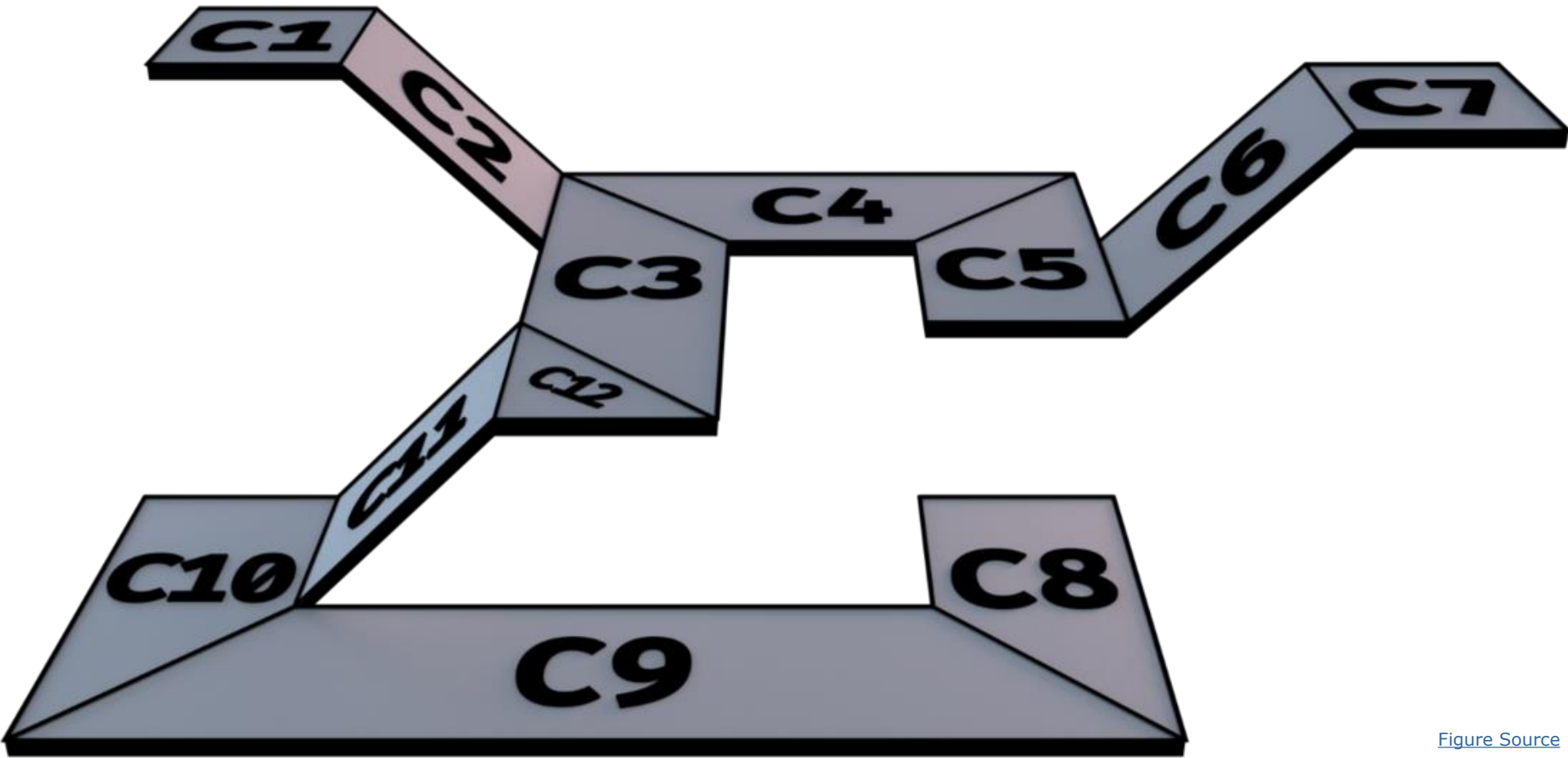
Pathfinding Demo

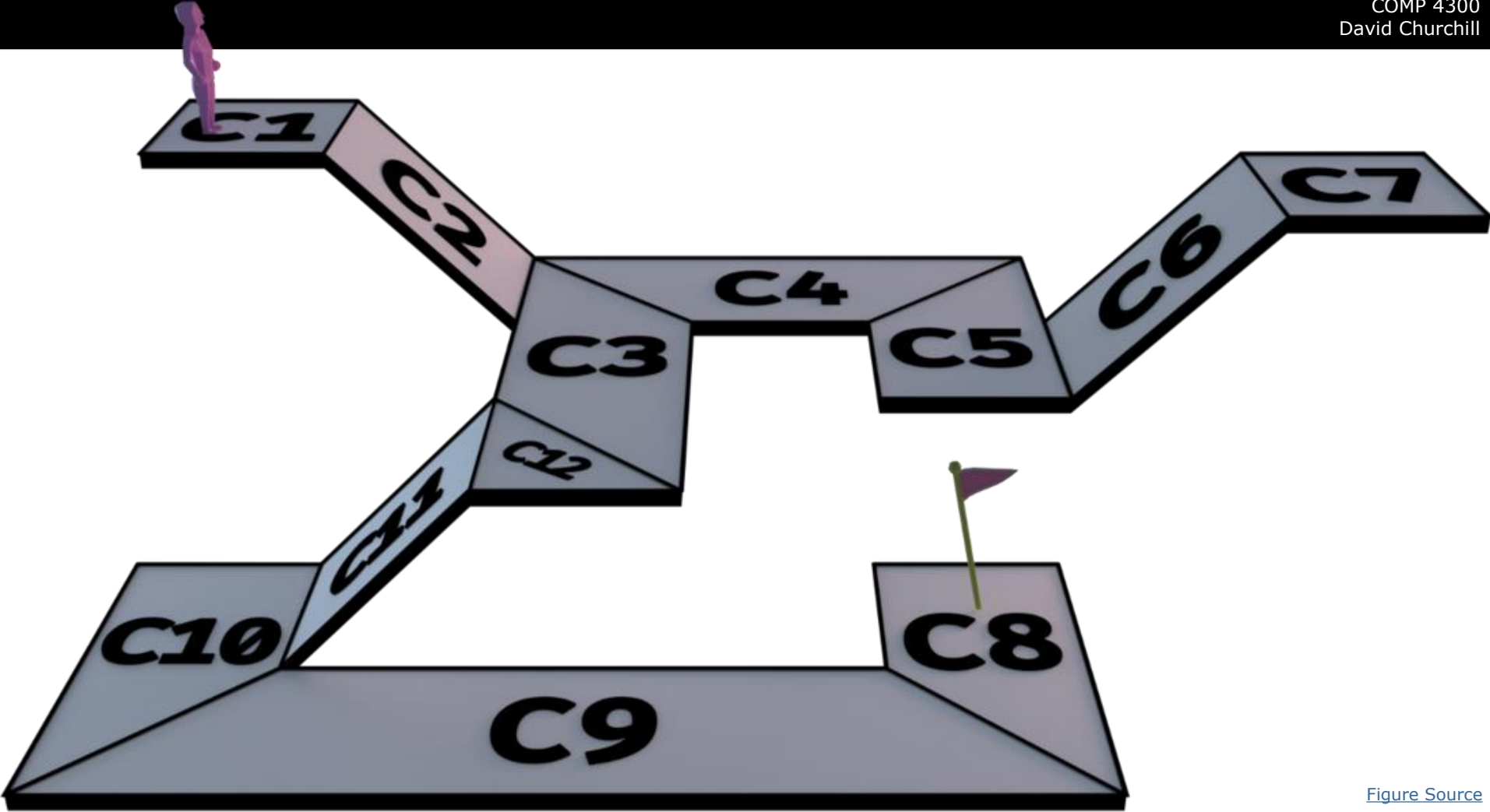
<http://www.cs.mun.ca/~dchurchill/search/>

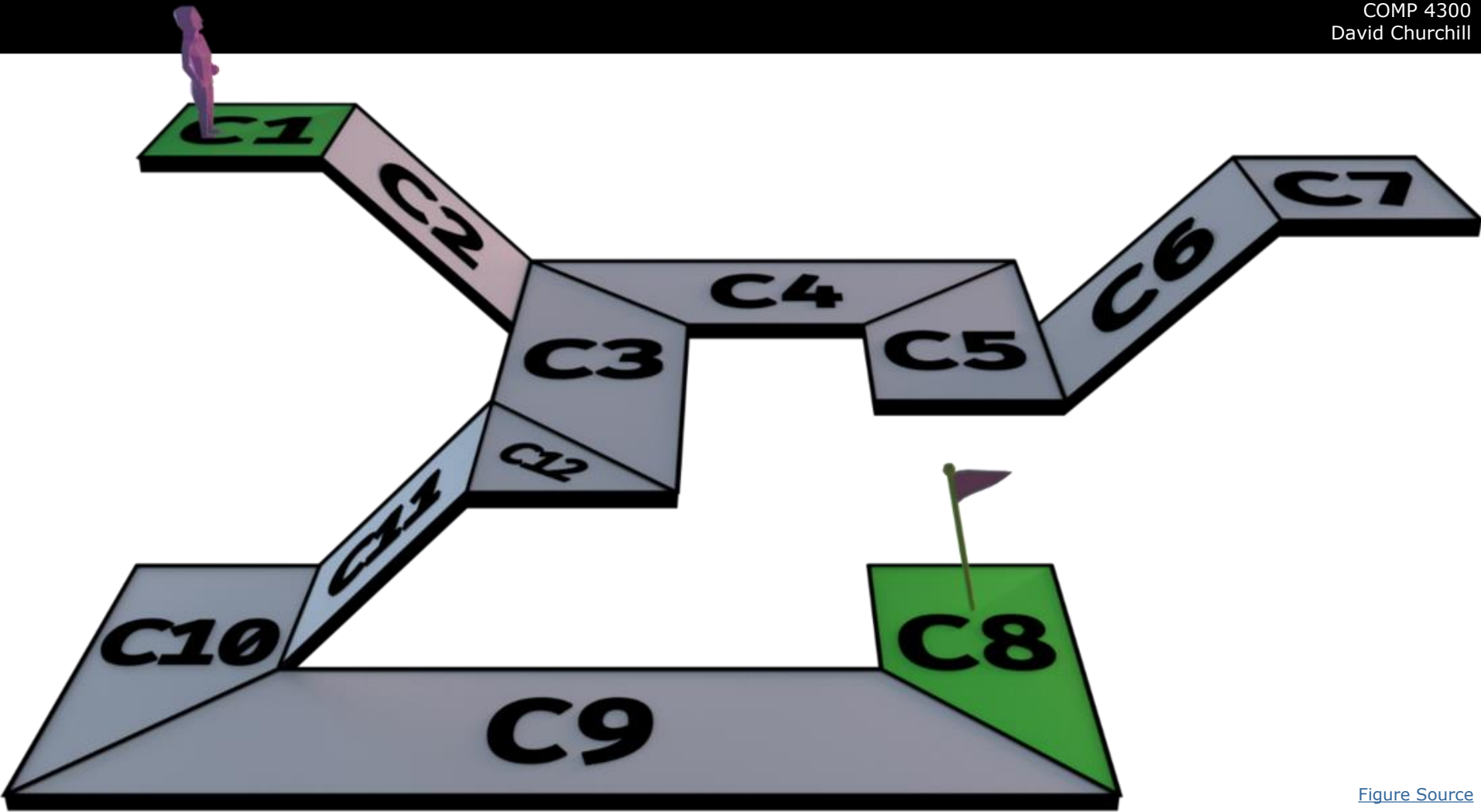
Path-finding Workflow

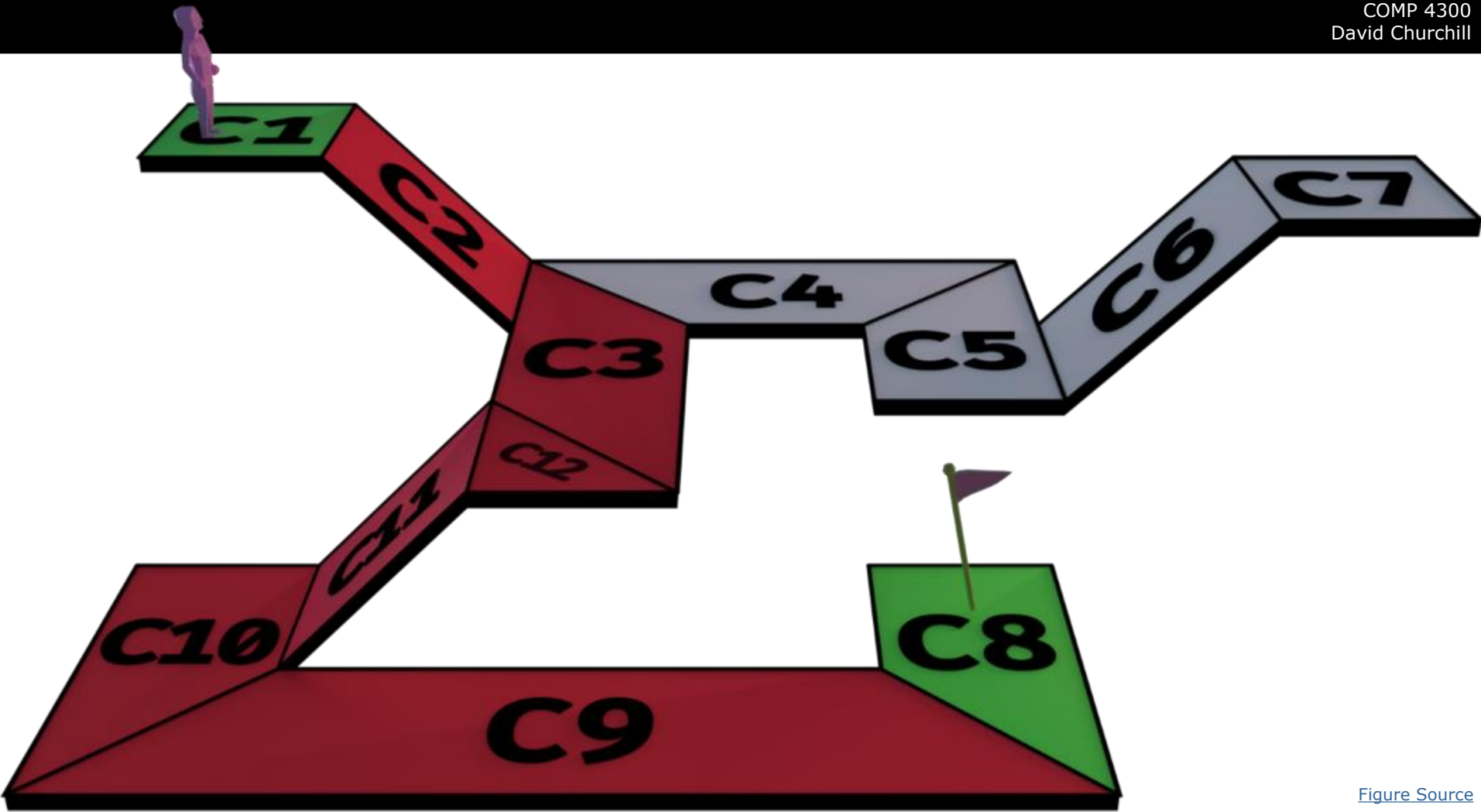
1. Identify on the problem you're solving
2. Construct a space representation
3. Decide which actions are possible
4. Be given a start / goal locations
5. Apply an algorithm which finds a path
6. Follow the path from start to goal

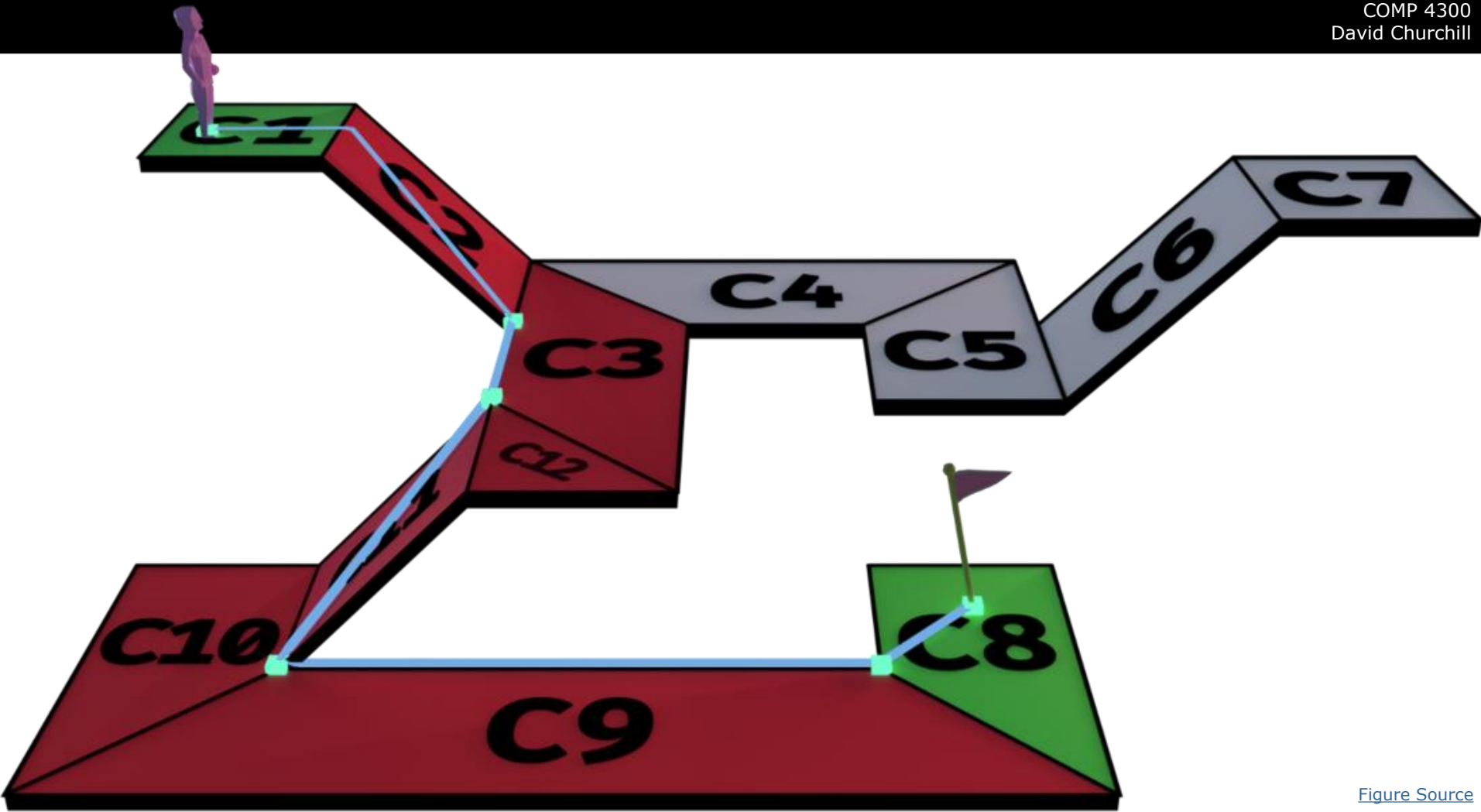


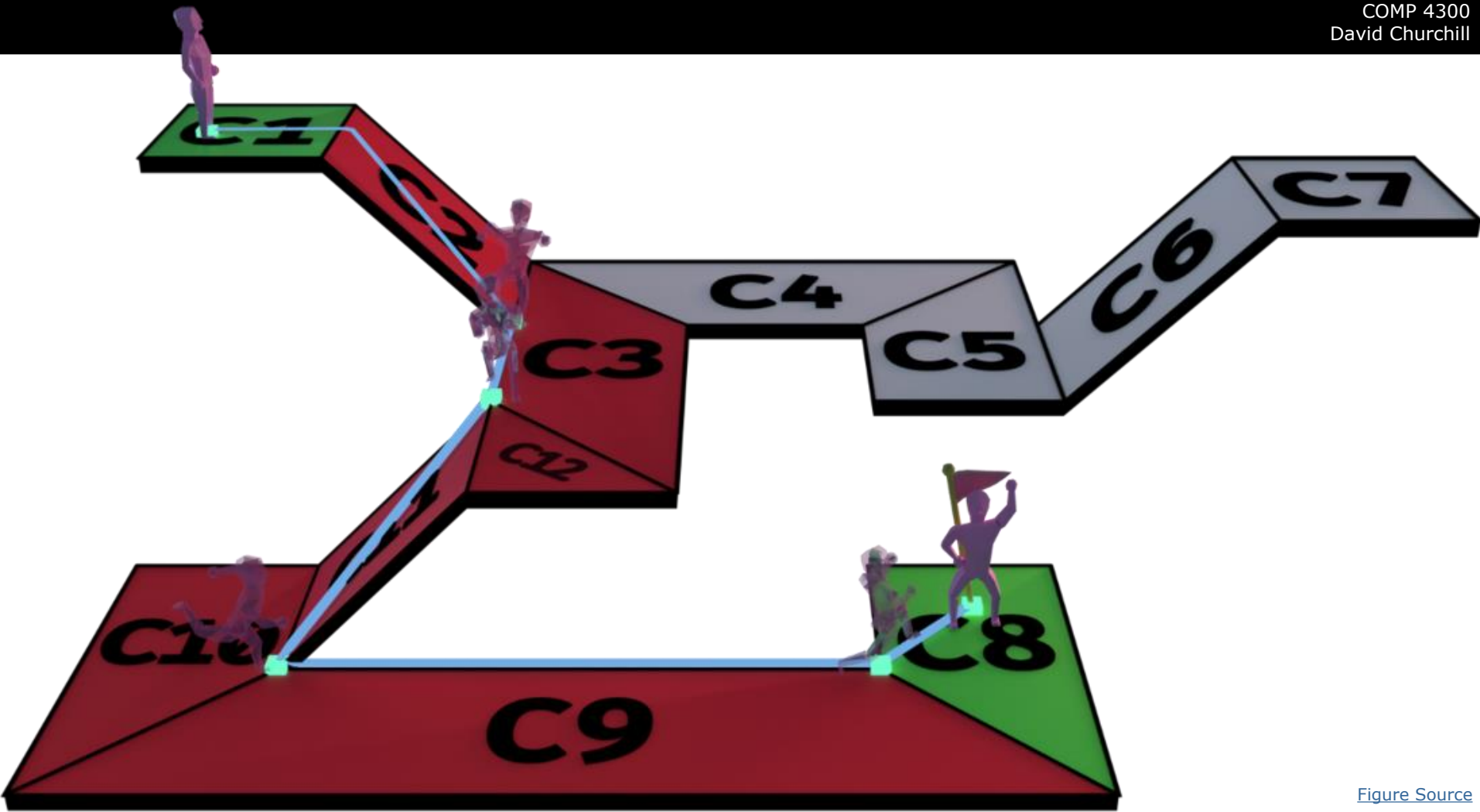






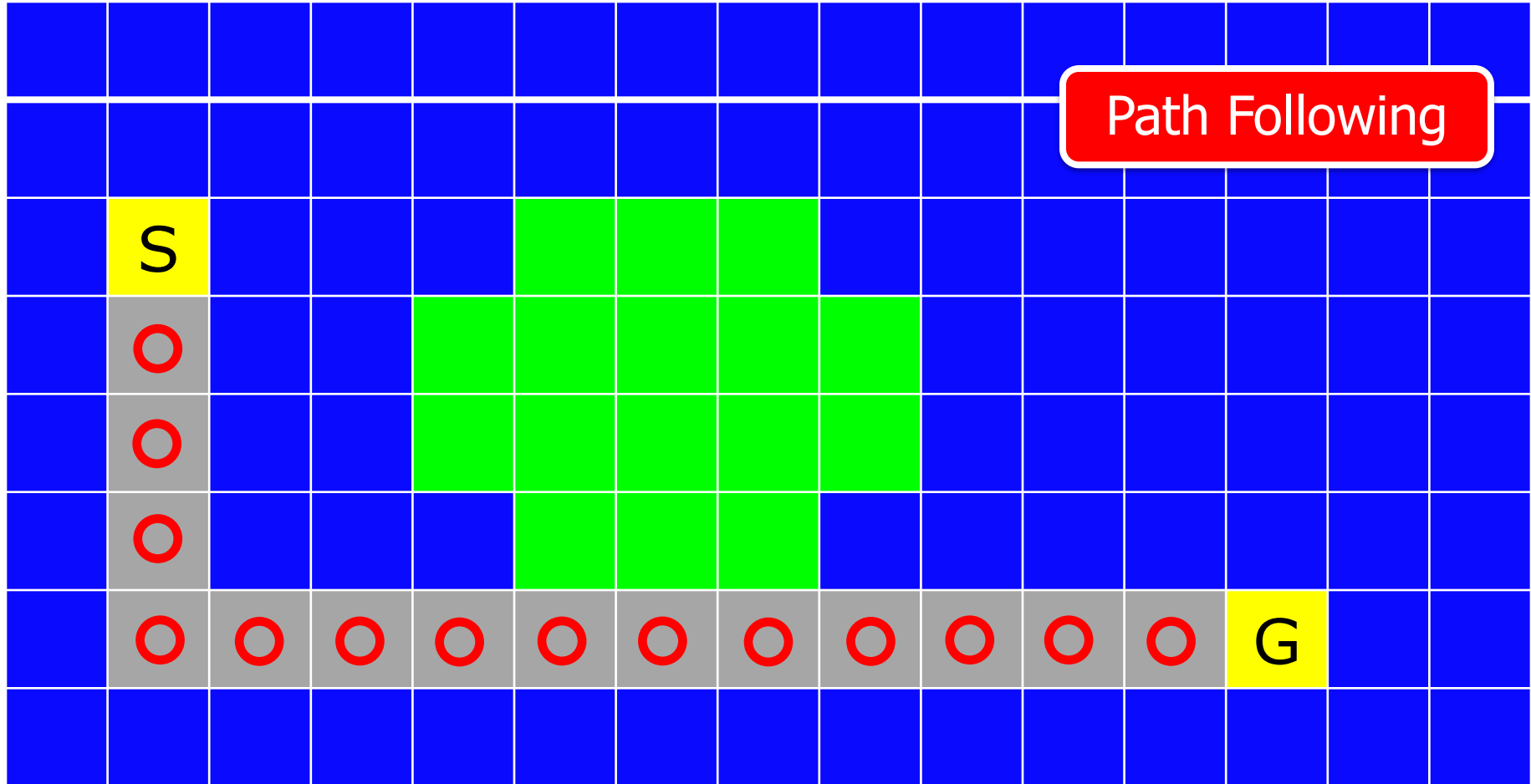


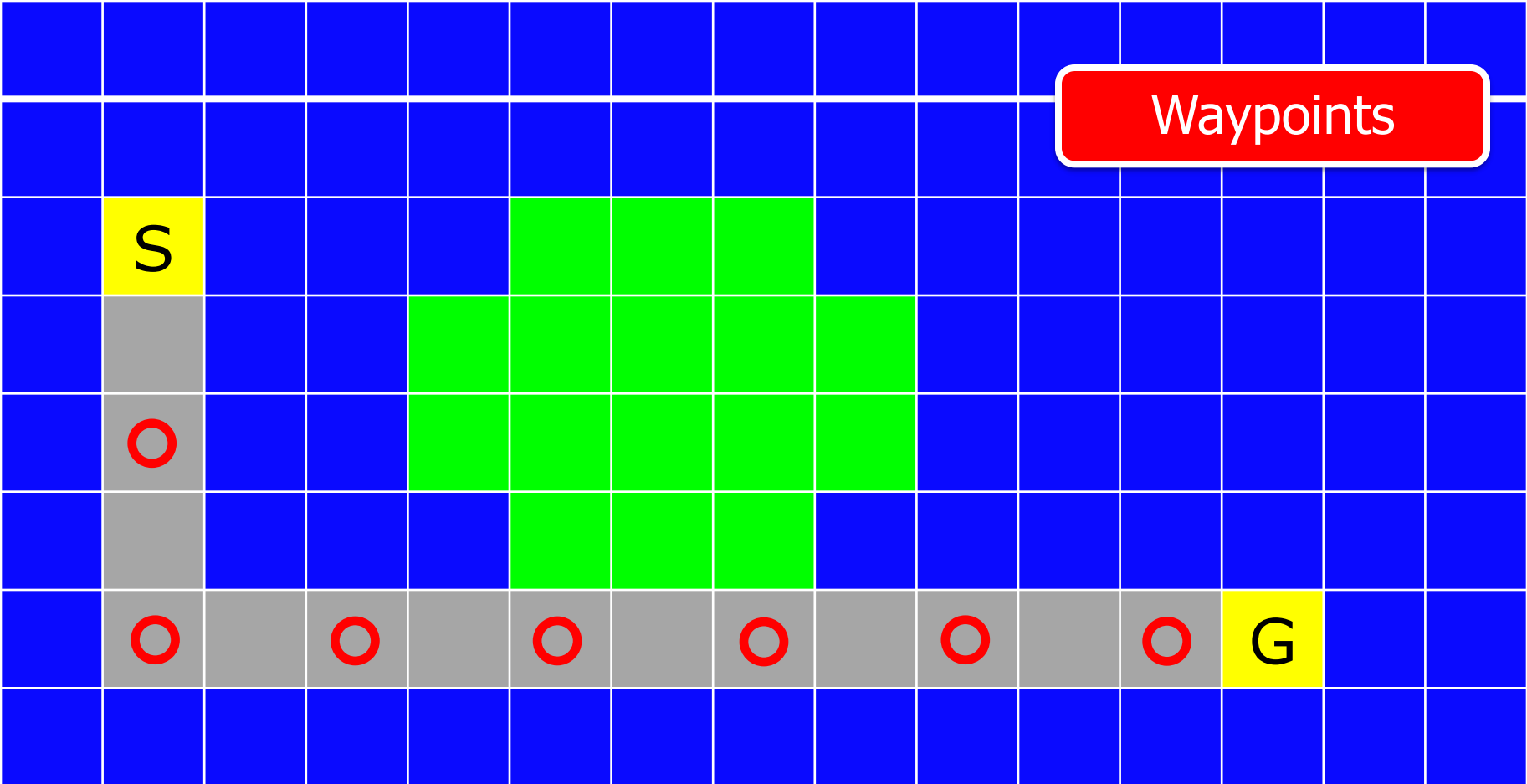


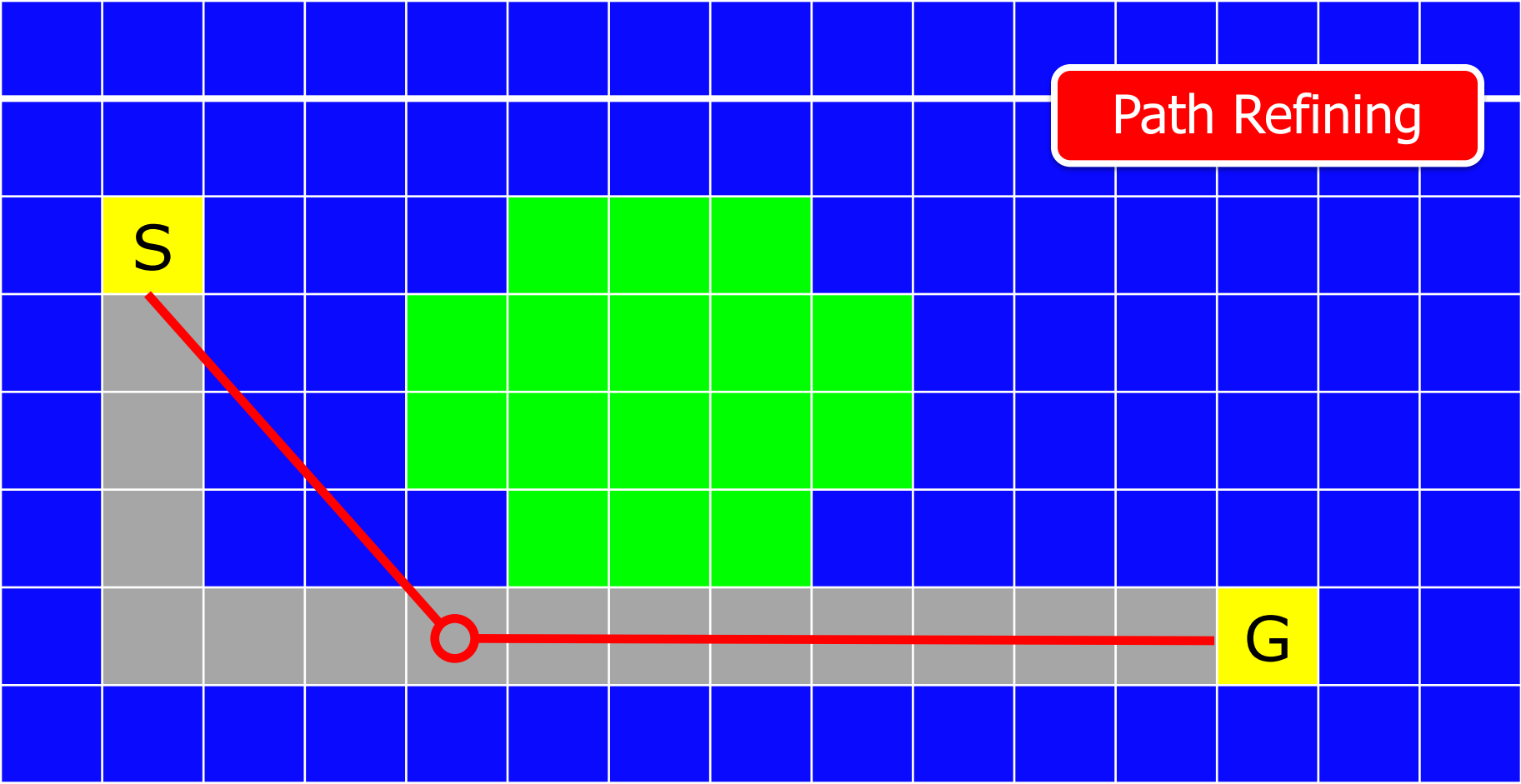


Path Following

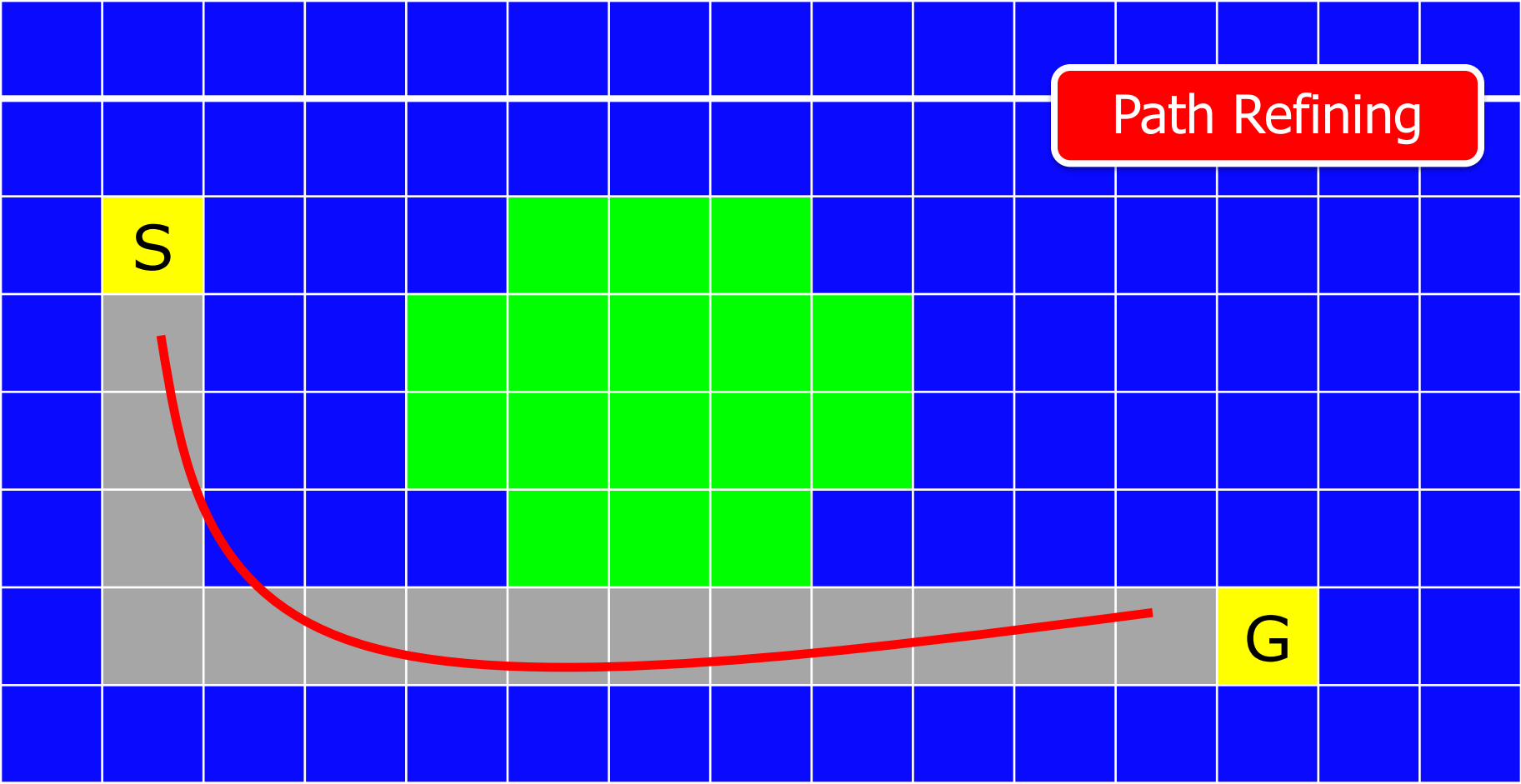
- Now that we have a path, how do we make our agent traverse it in the game?
- Simplest way: waypoints
 - Each point along the path is a waypoint
 - Path to the next waypoint is always 'clear'
 - Walk from one waypoint to the next
 - Stop when NPC reaches the goal (out of wp)





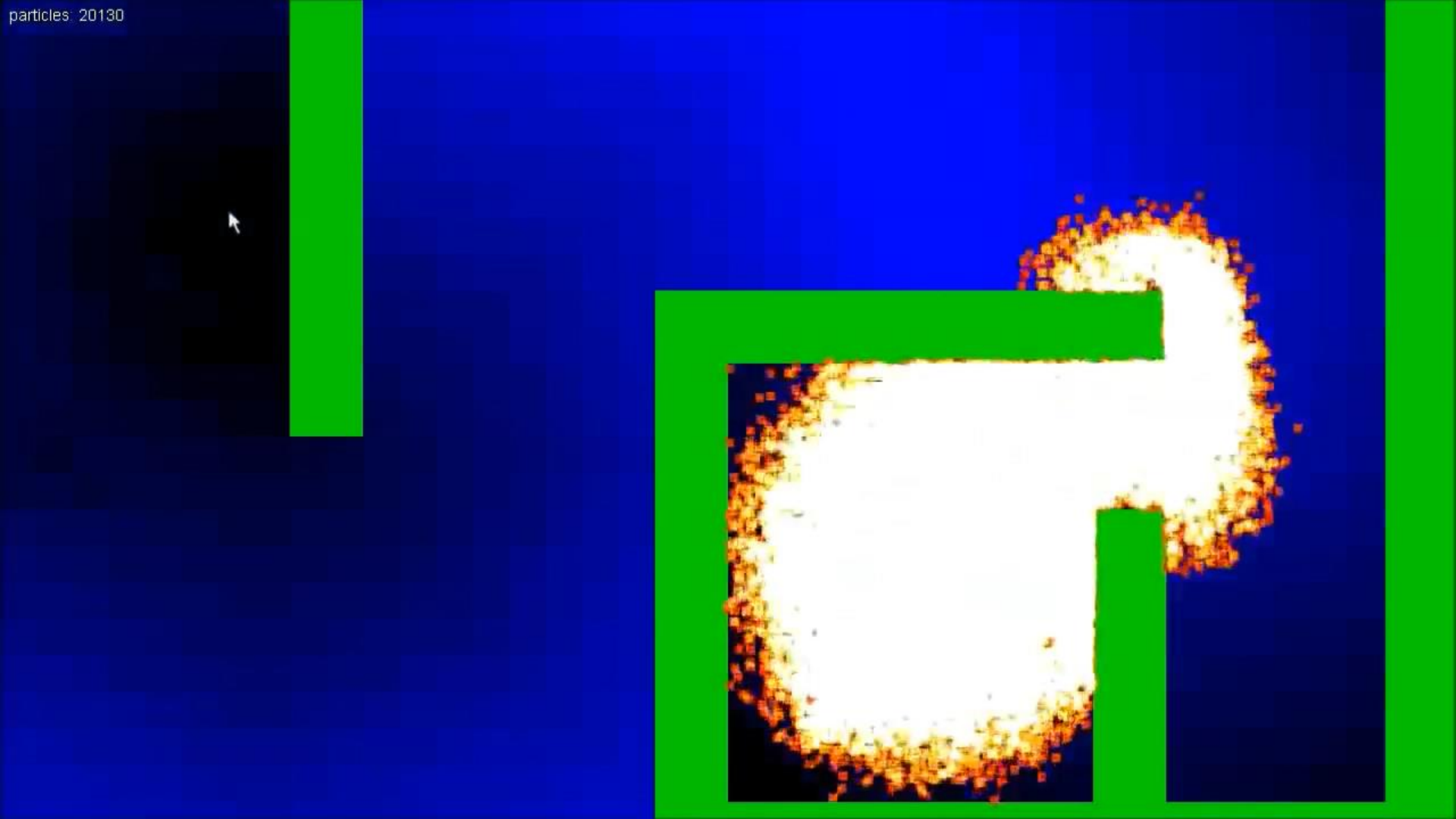


Path Refining

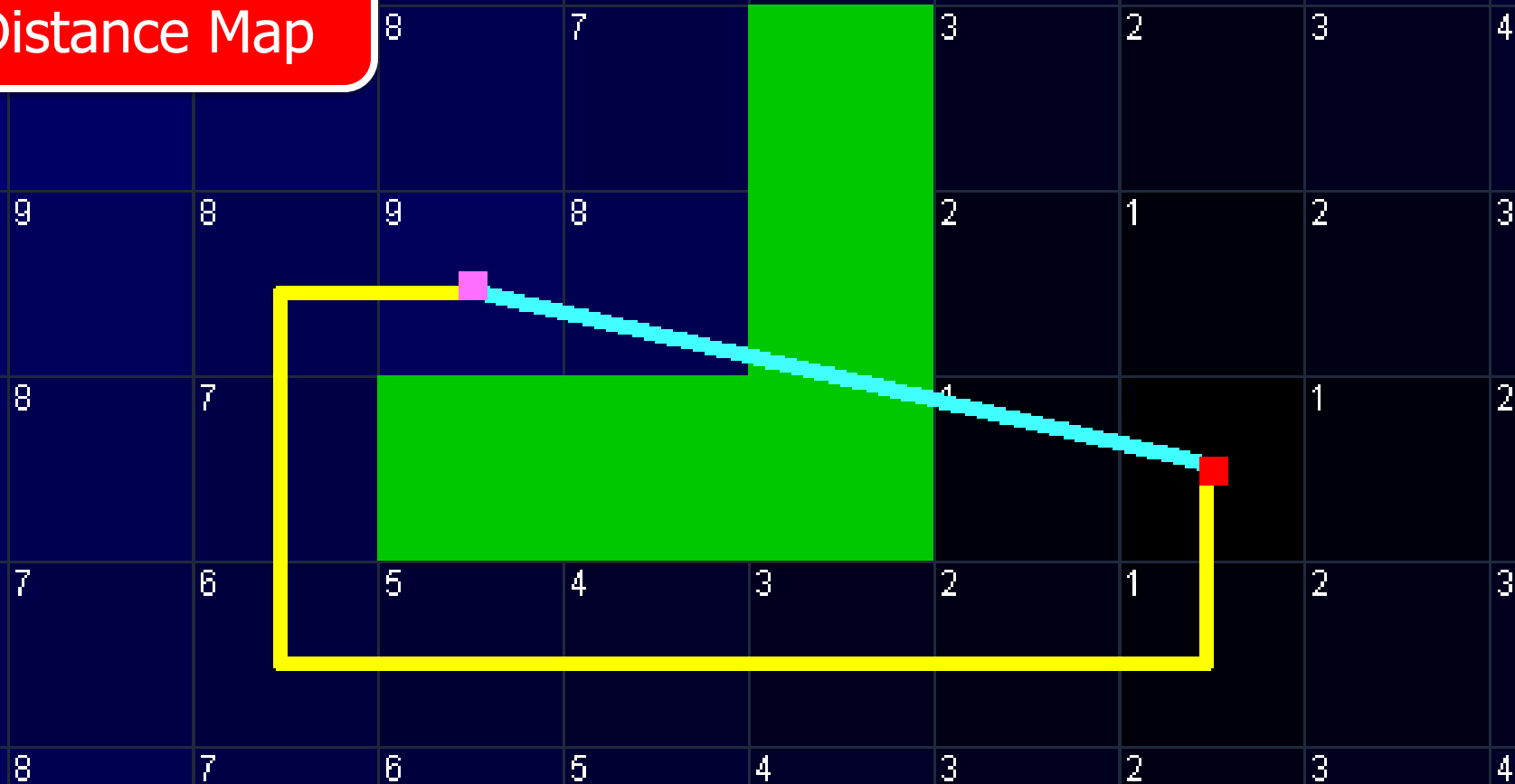


Vector Fields for Pathfinding

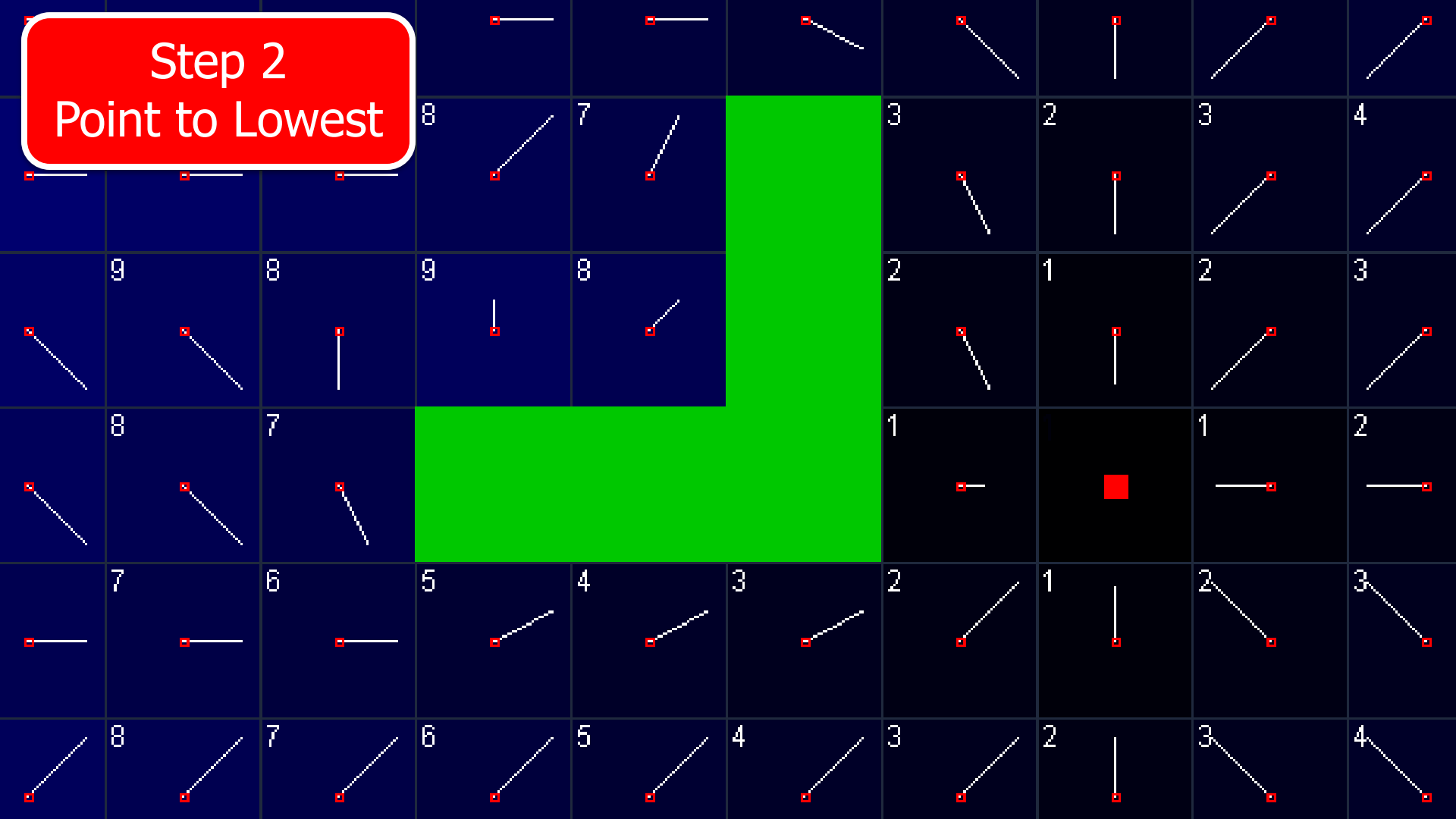
- Flow fields, influence maps, vector maps
- Very efficient structure for path-finding in case of 'many paths to single goal'
- Any case where multiple NPCs need to go to a single location, consider using these
- Must balance computation time with how many NPCs, etc

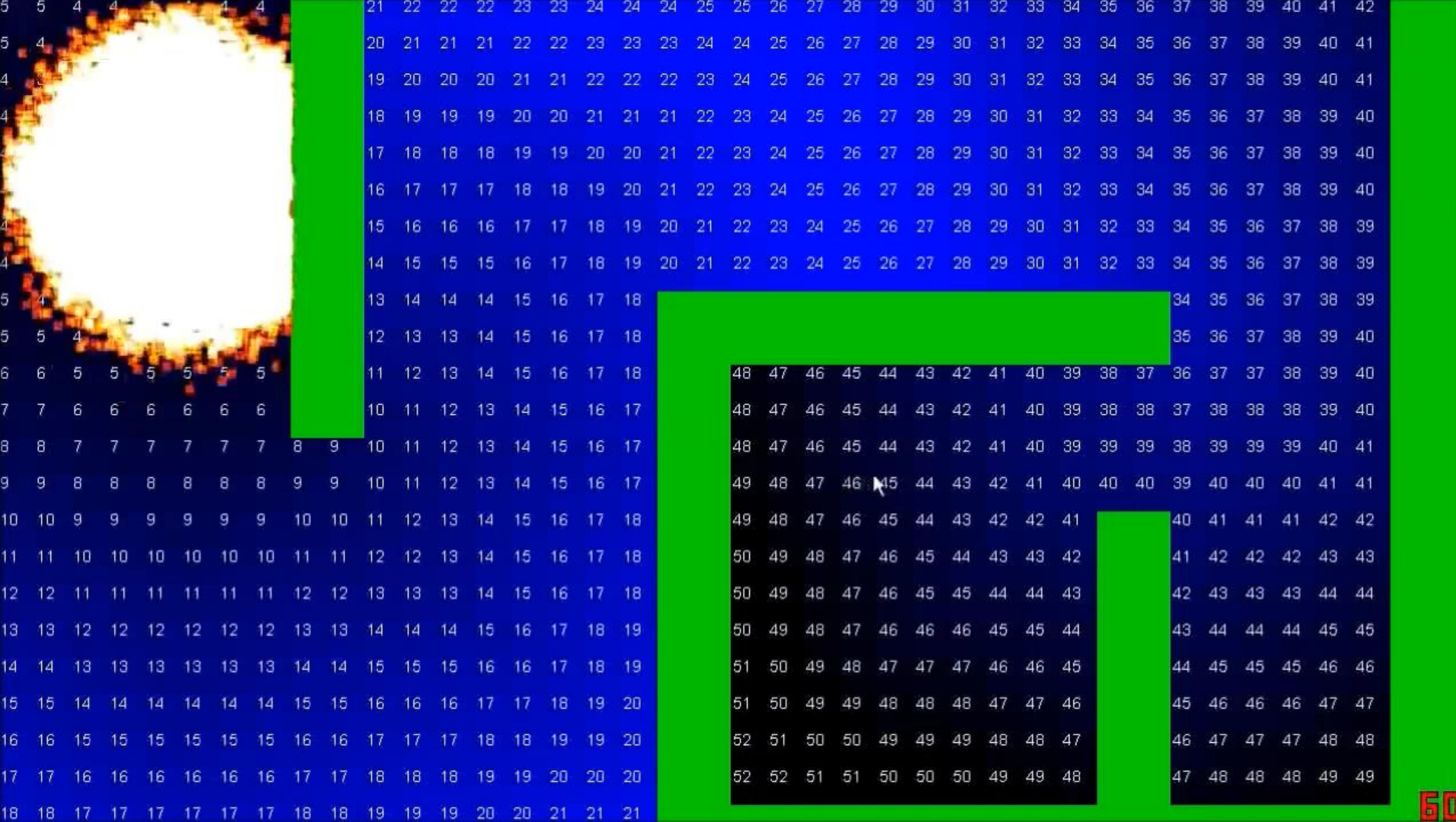


Step 1 Distance Map



Step 2 Point to Lowest





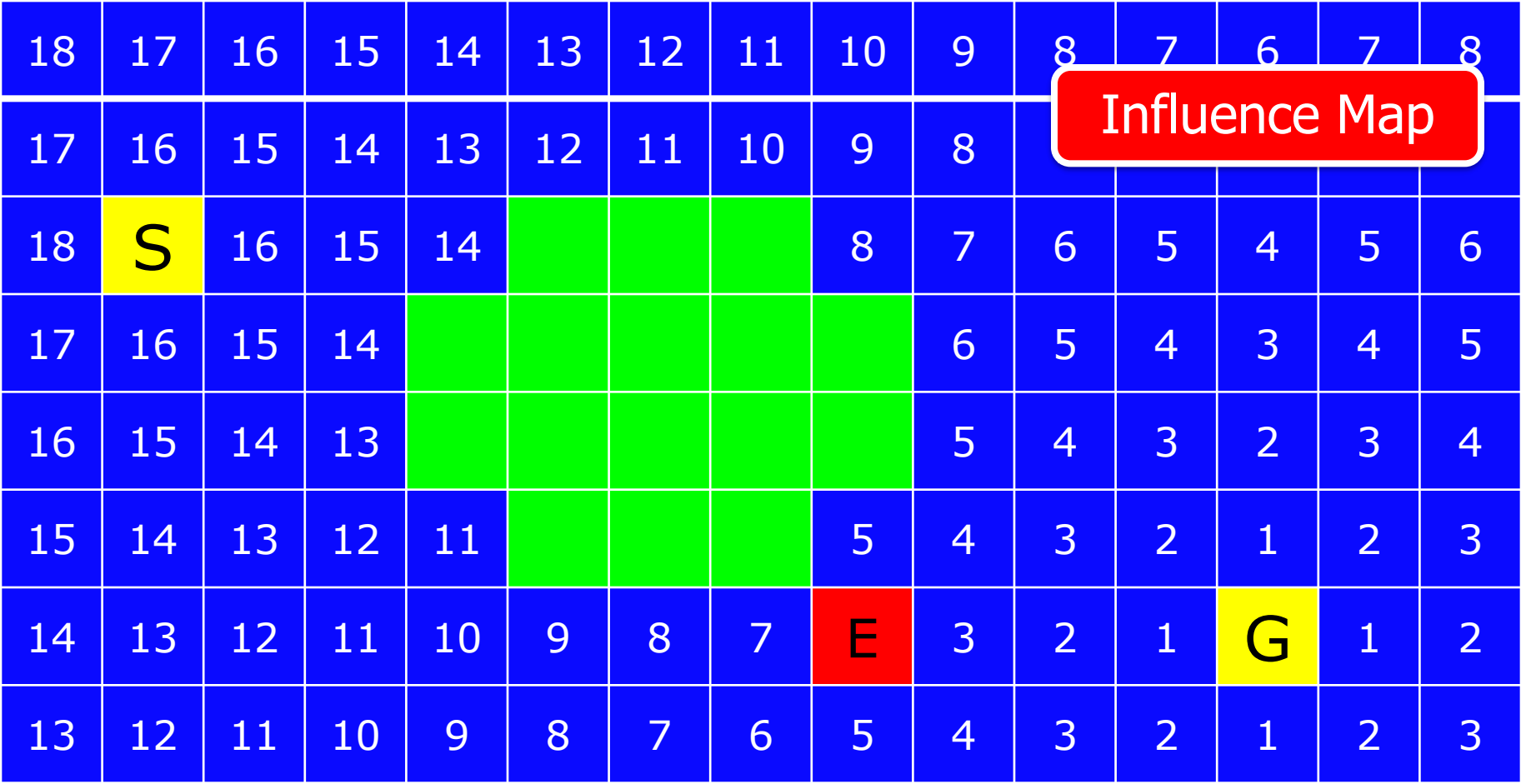
Vector Fields

- Last 4 Slides:
 - <https://gamedevelopment.tutsplus.com/tutorials/understanding-goal-based-vector-field-pathfinding--gamedev-9007>

Influence Maps

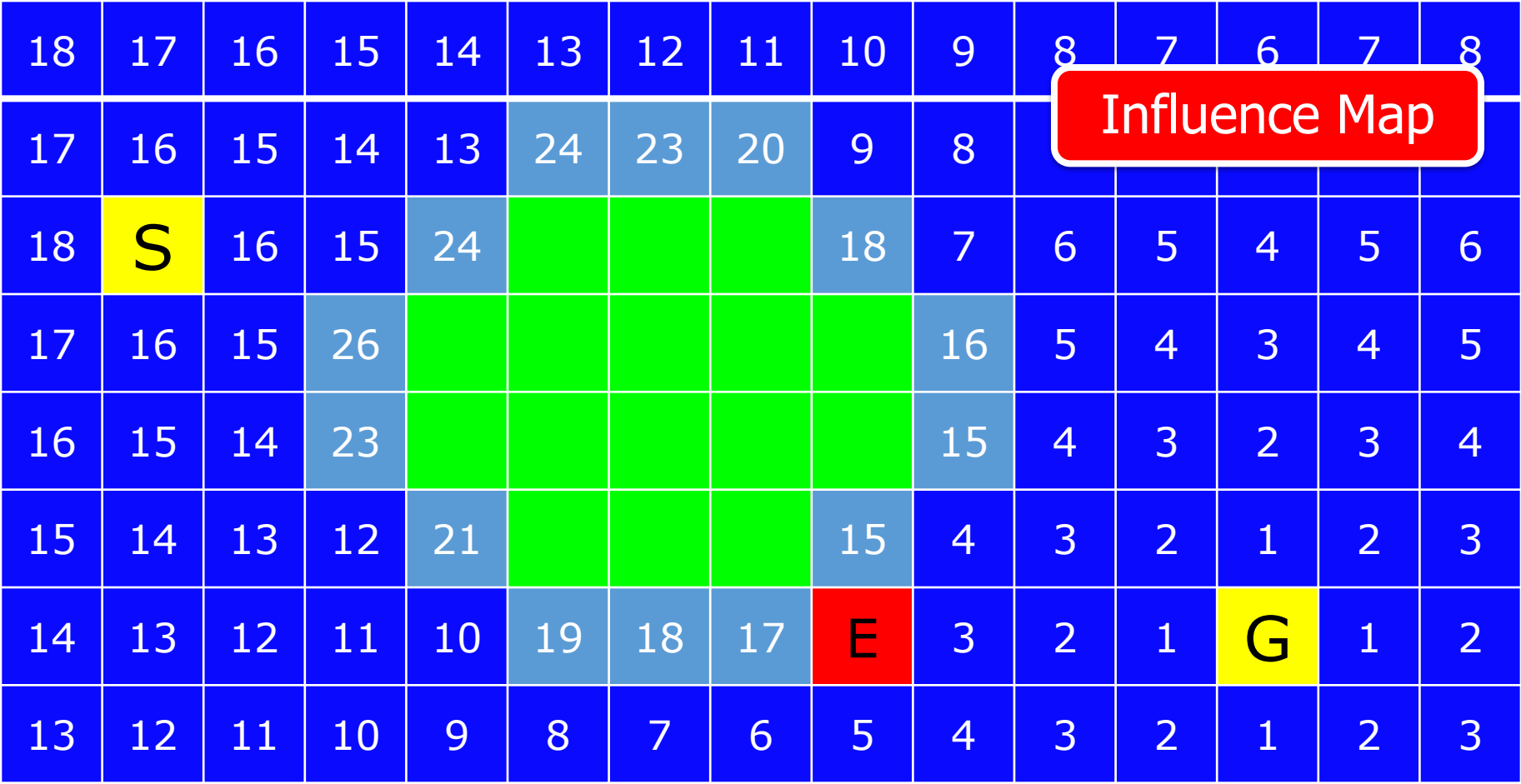
- Influence maps typically incorporate other logic than simple distance to goal
- Used when difficult to come up with an objective function for other algorithms
- Some examples include:
 - Distance to attacking enemies
 - Distance from walls / obstacles

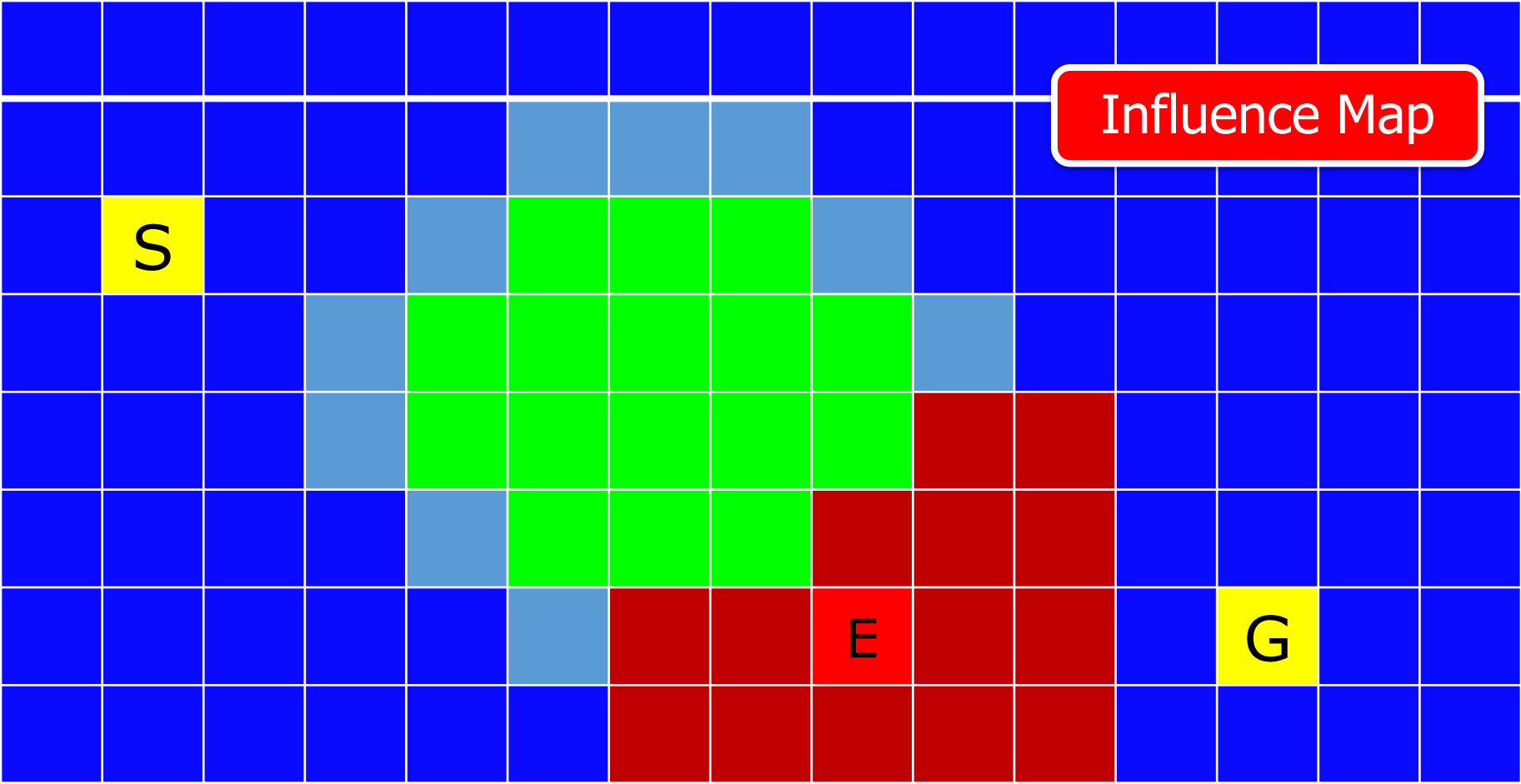




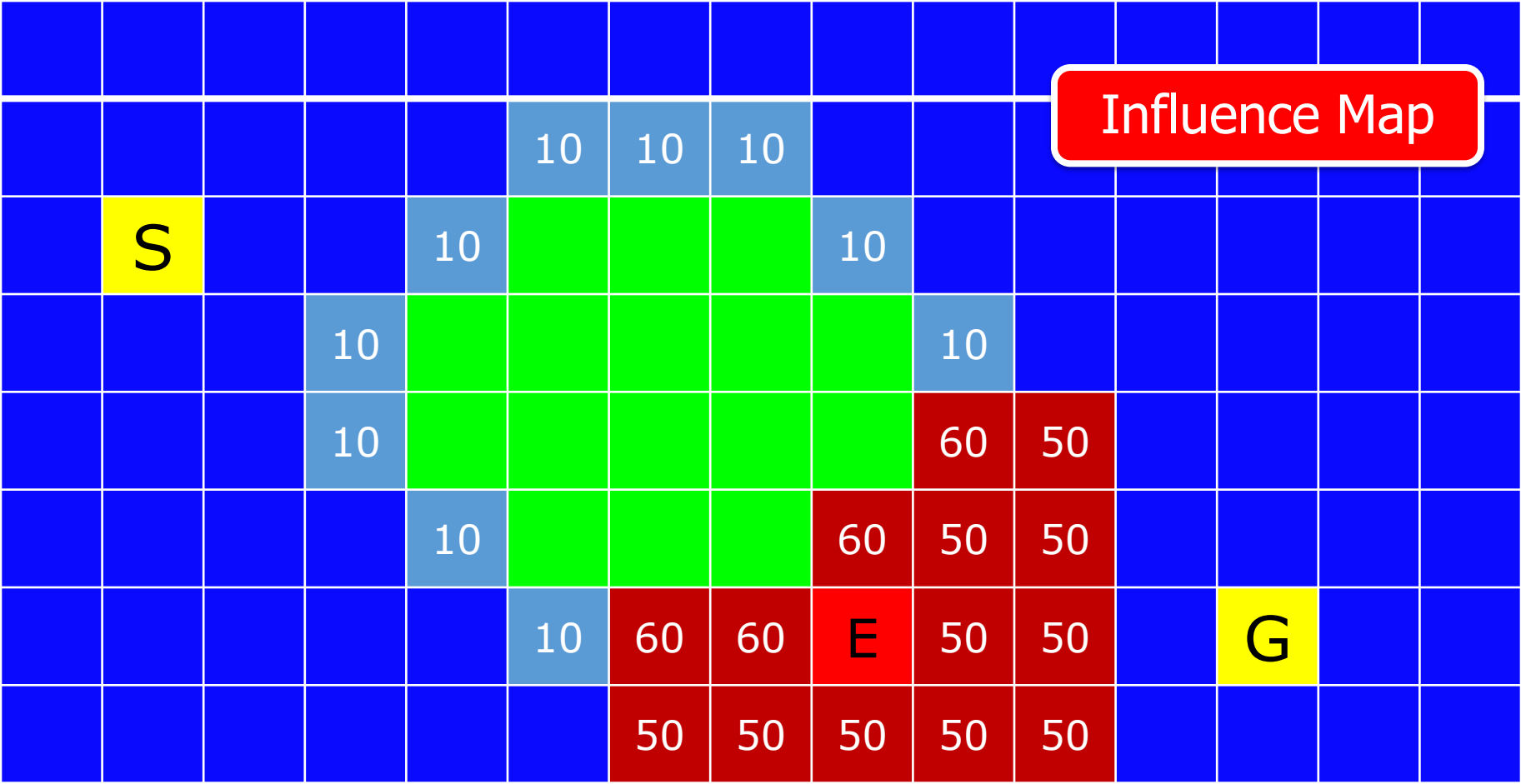


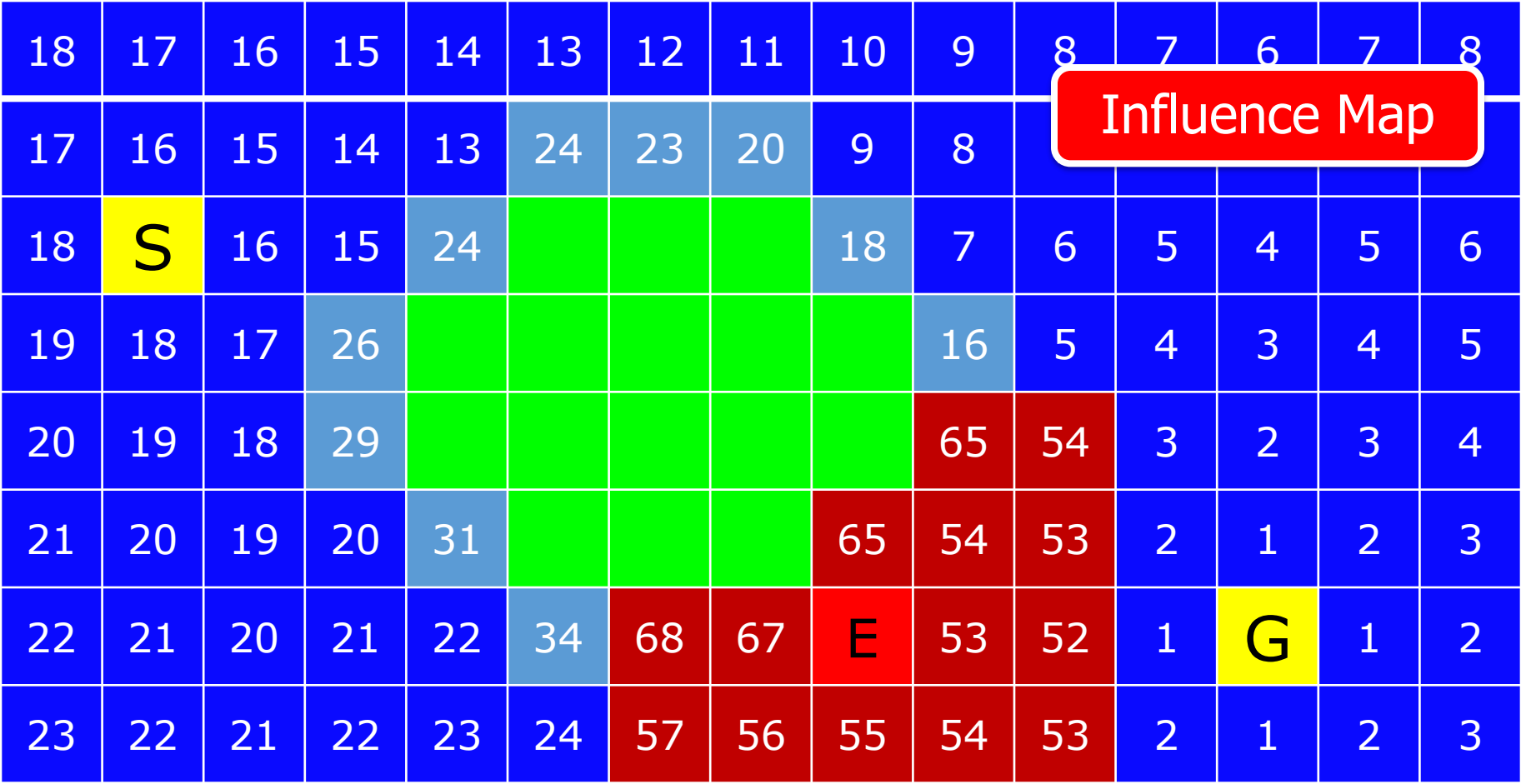






Influence Map





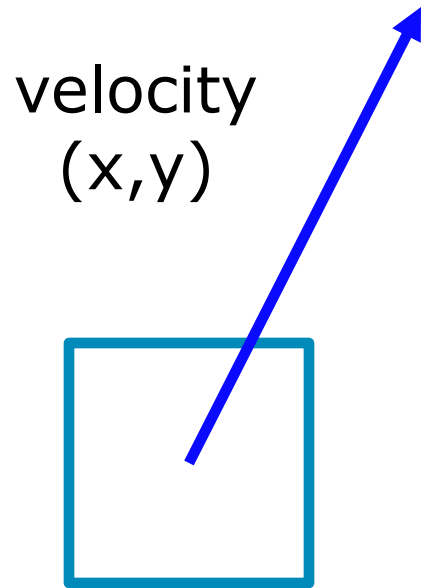
Steering Behaviors

- Steering can be seen as a **framework** for controlling autonomous agents such as robots, CGI in movies, NPCs in games
- Steering adds more **realism** and **fluidity** to agent movement, can enhance experience
- Steering can be really fun to implement, play with **parameters**, and see what effects can be produced

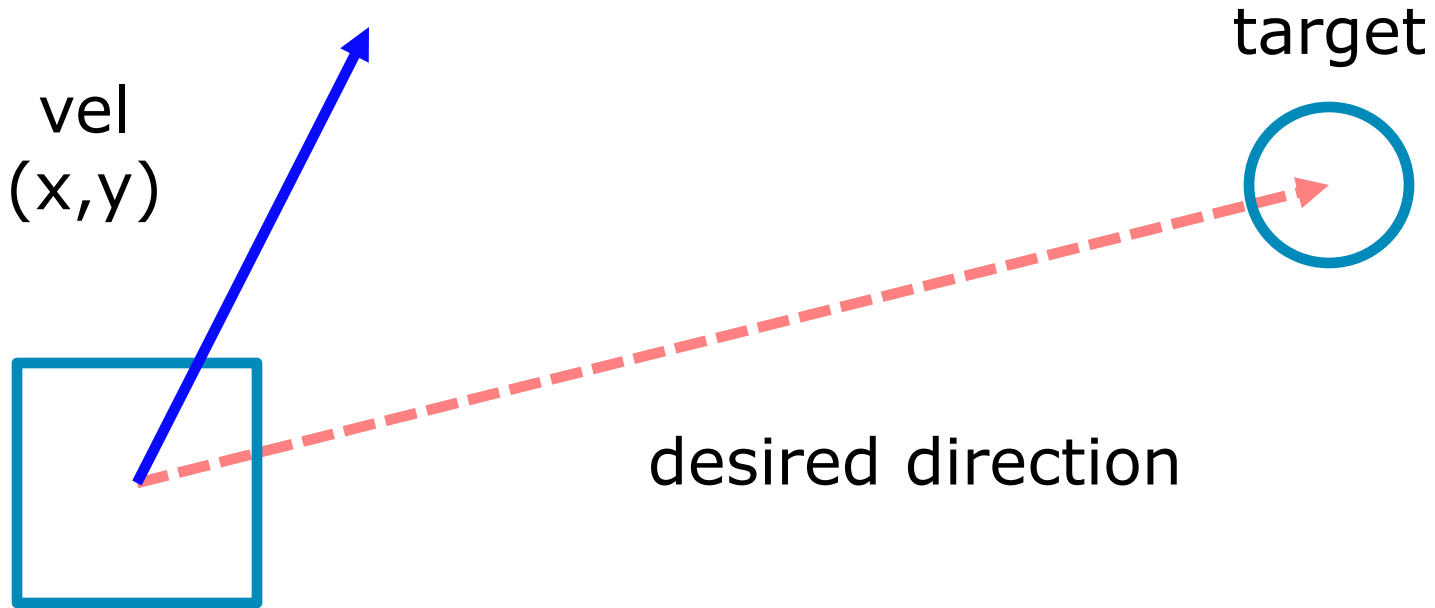
Steering Behavior

- At its core, steering **smooths the movement** of Entities in games
- Changing directions instantaneously or 'stop on a dime' can look bad in games
- Steering acts as a sort of '**force**' that smoothly guides an Entity toward a target
- Will be part of the project

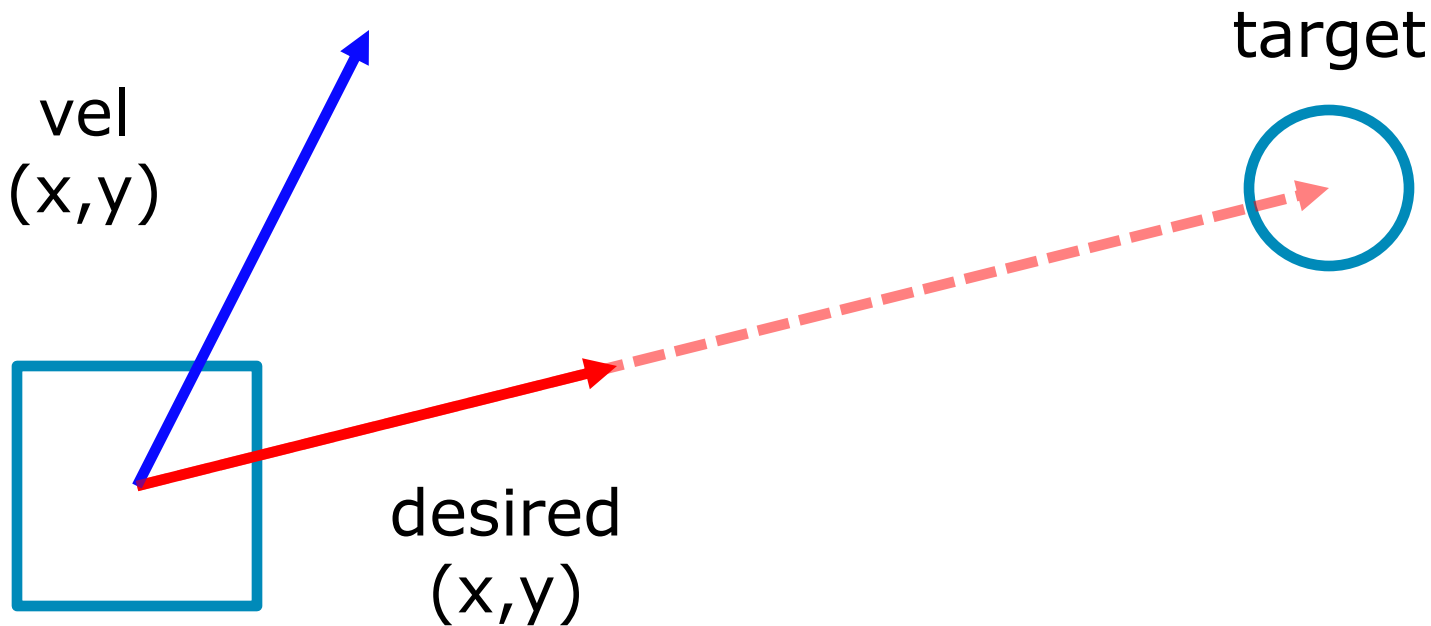
Steering Behaviors



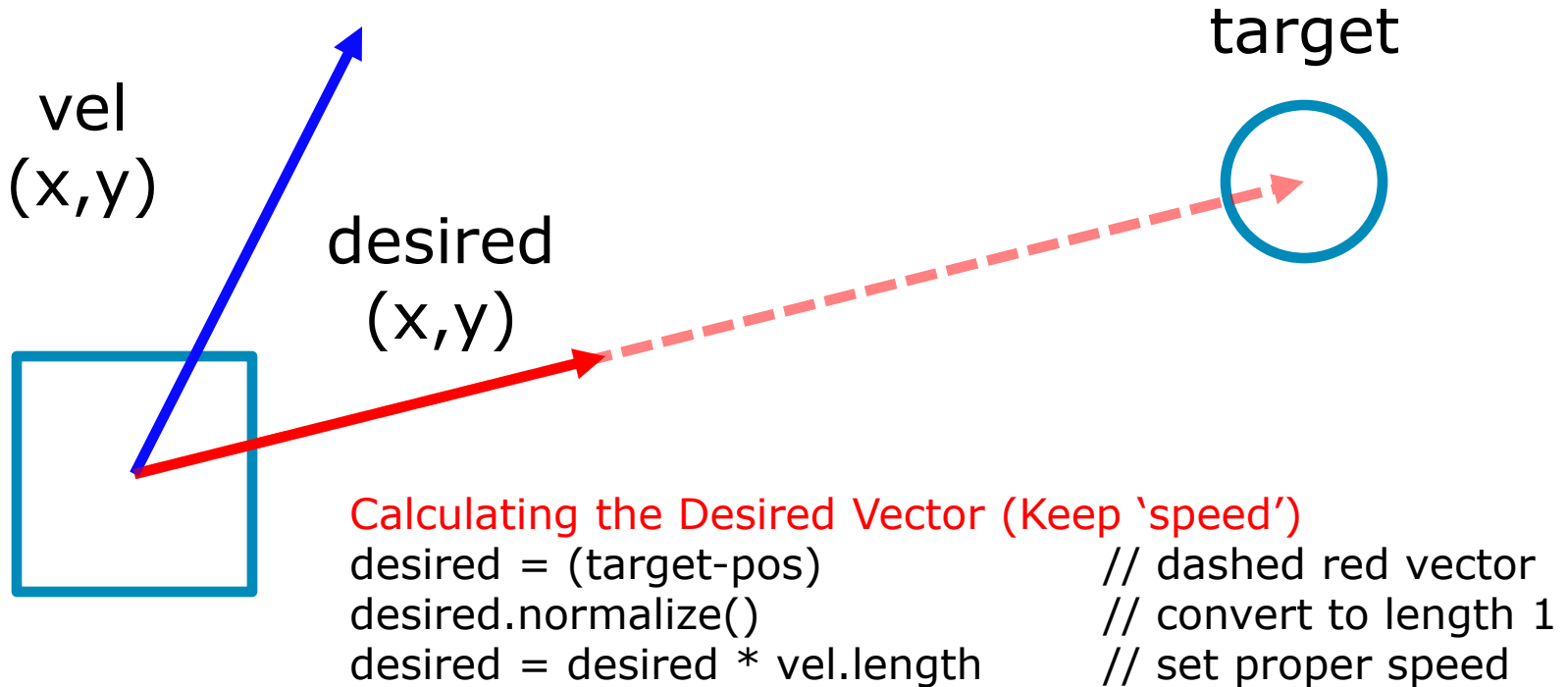
Steering Behaviors



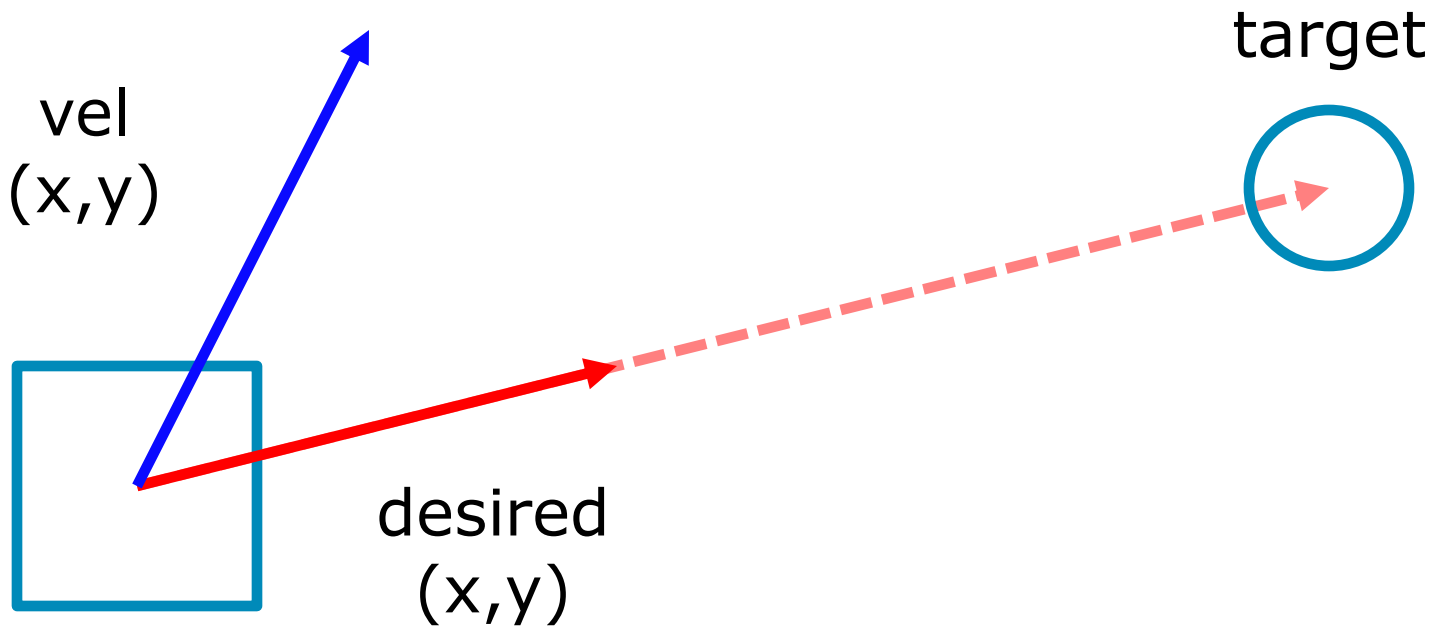
Steering Behaviors



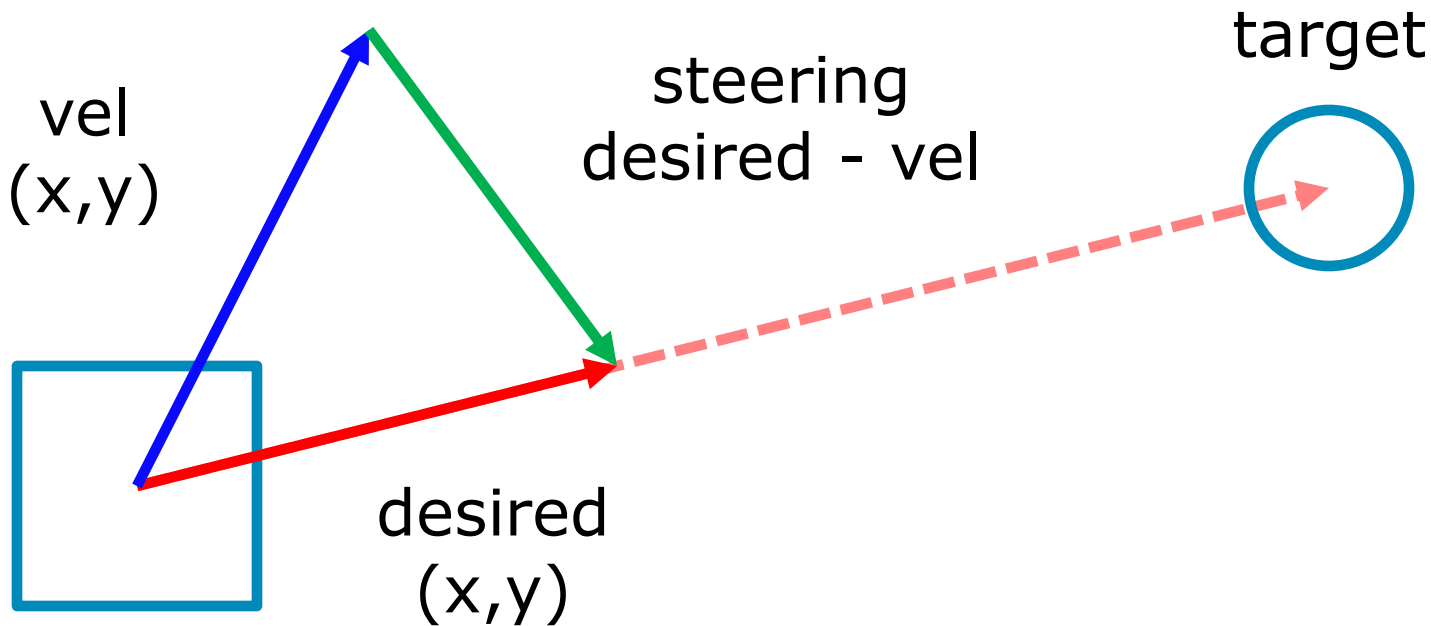
Steering Behaviors



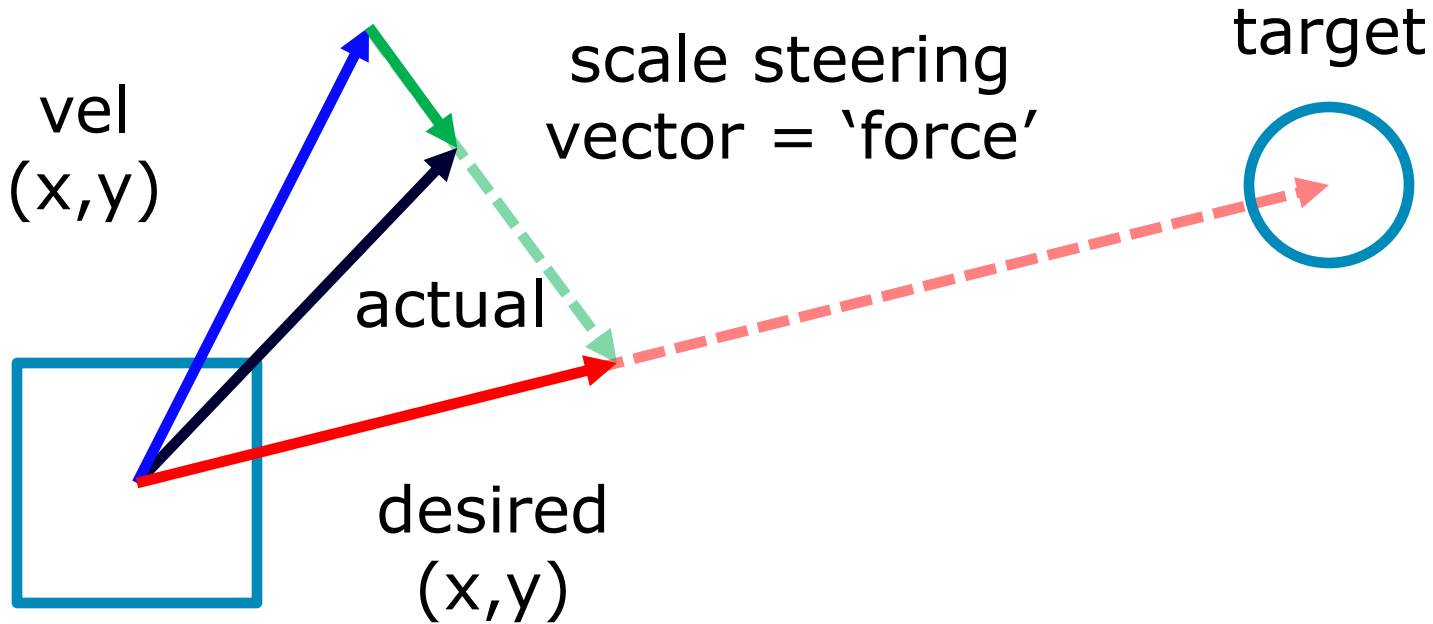
Steering Behaviors



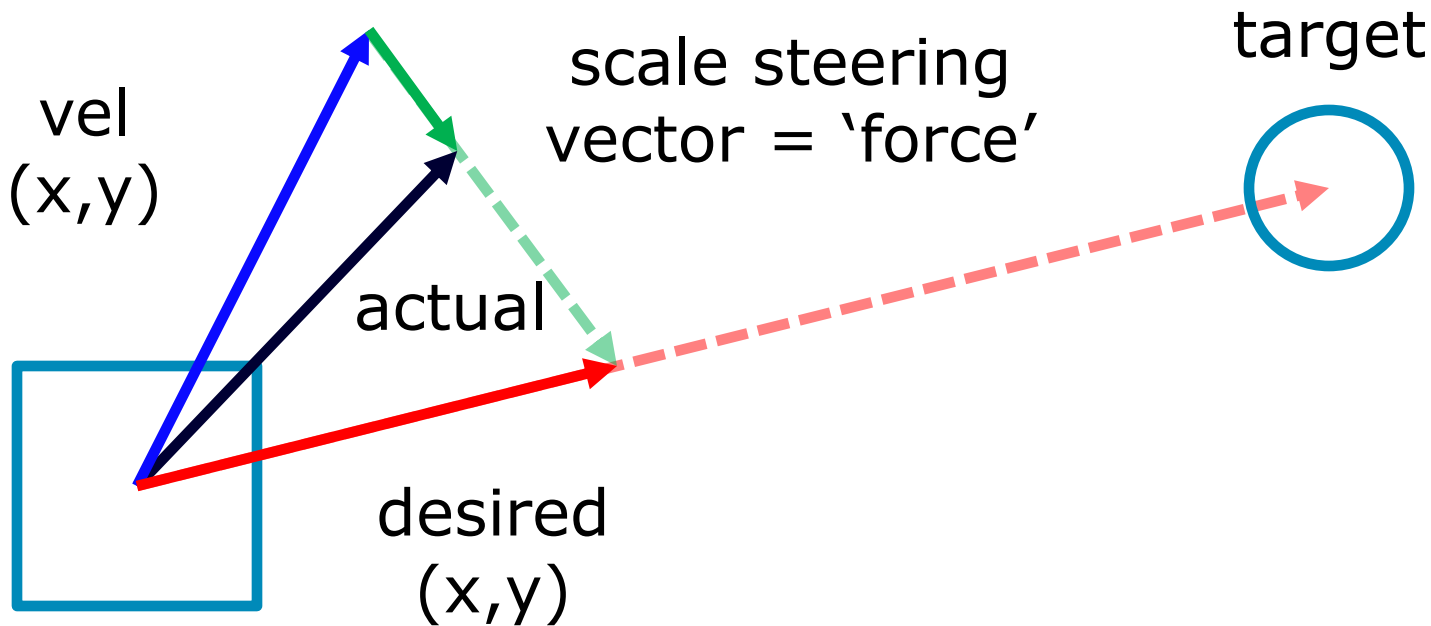
Steering Behaviors



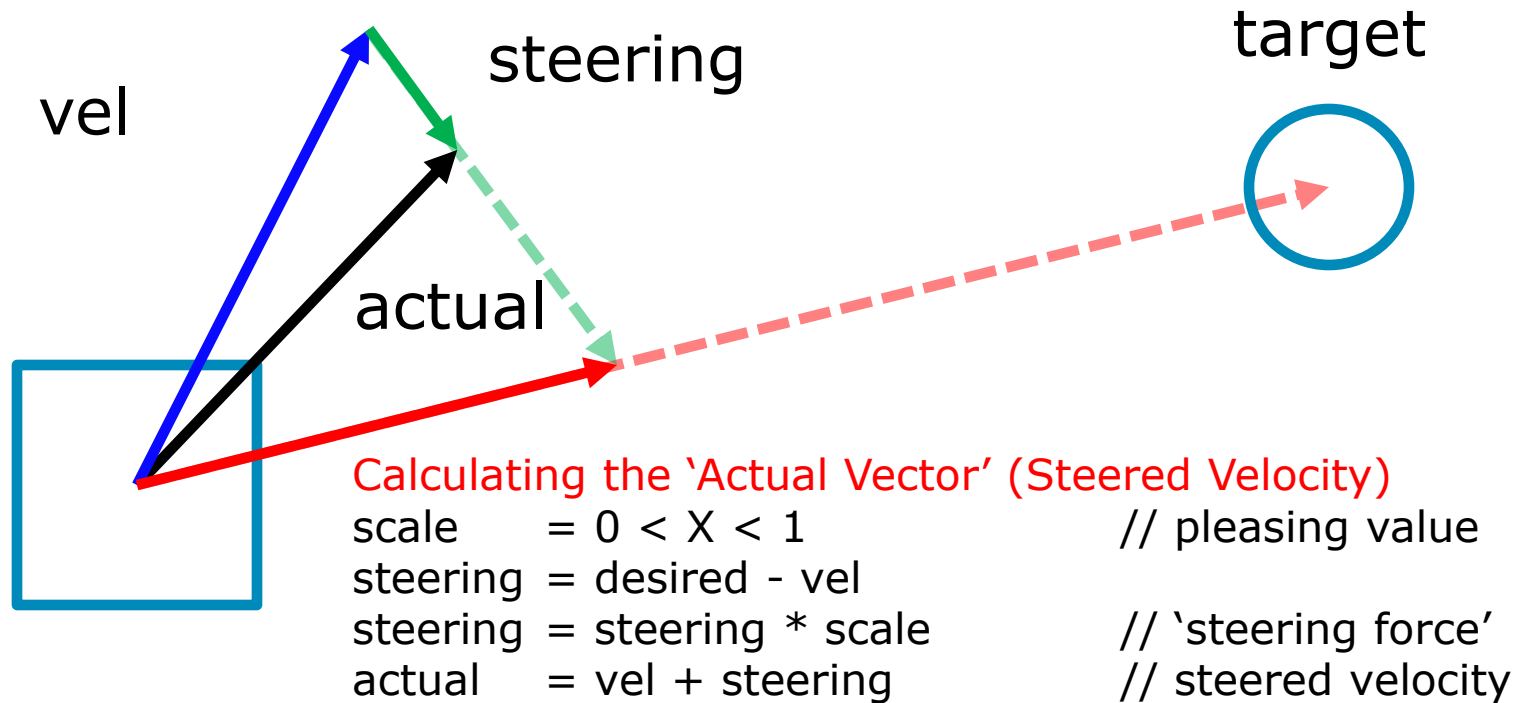
Steering Behaviors



Steering Behaviors

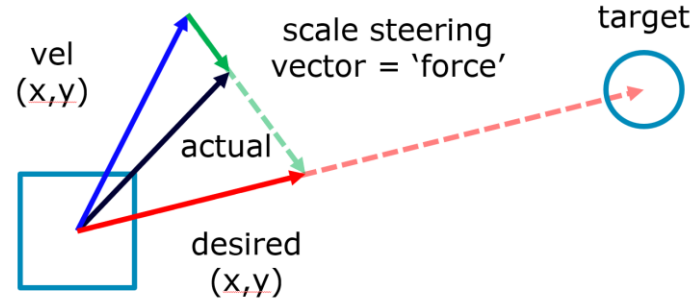


Steering Behaviors



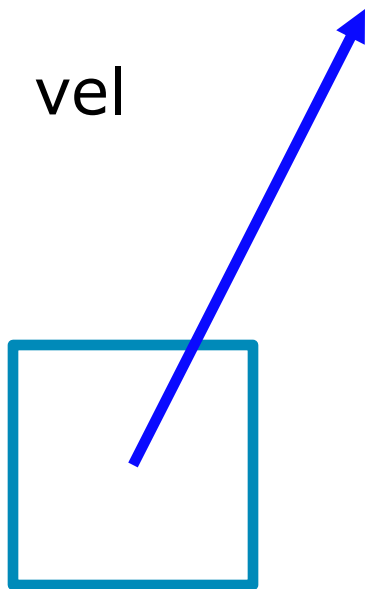
Steering Math

Given: `vel`, `target`, `scale`

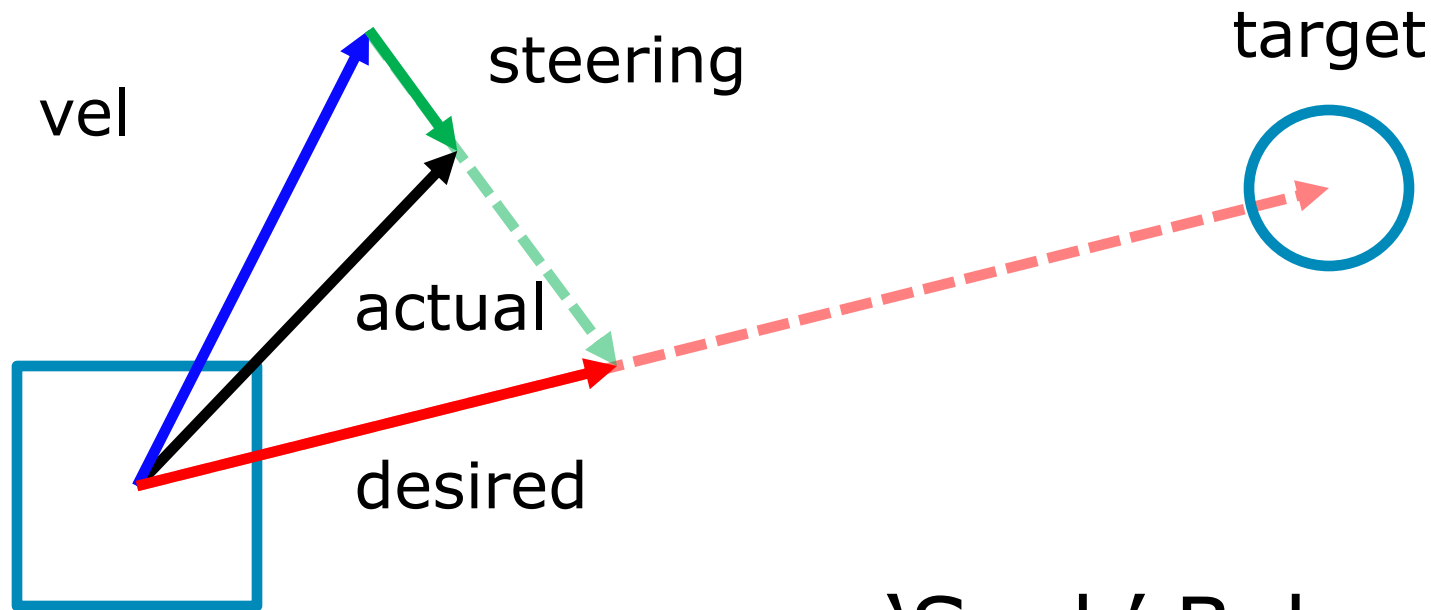


1. `desired = target - pos`
2. `desired = desired / desired.length`
3. `desired = desired * vel.length`
4. `steering = desired - vel`
5. `steering = steering * scale`
6. `actual = vel + steering`

Types of Steering

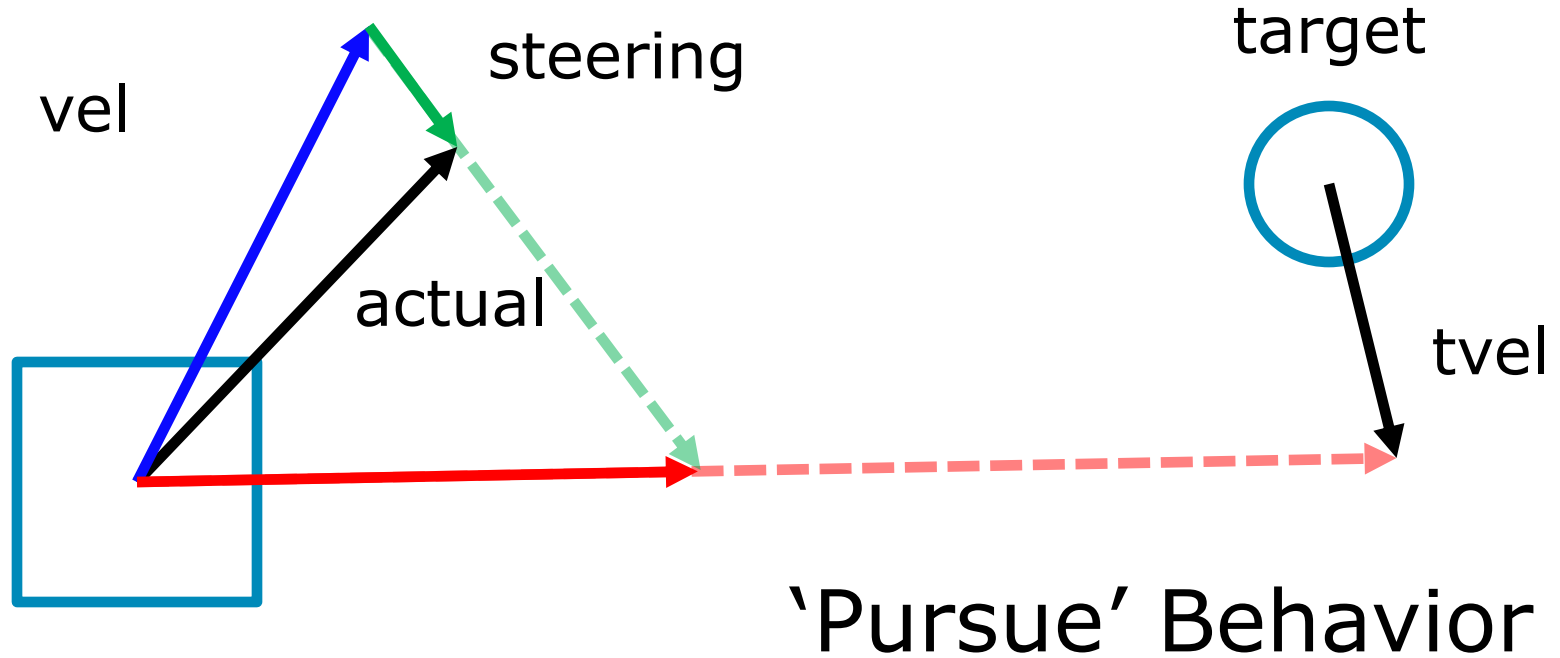


Types of Steering

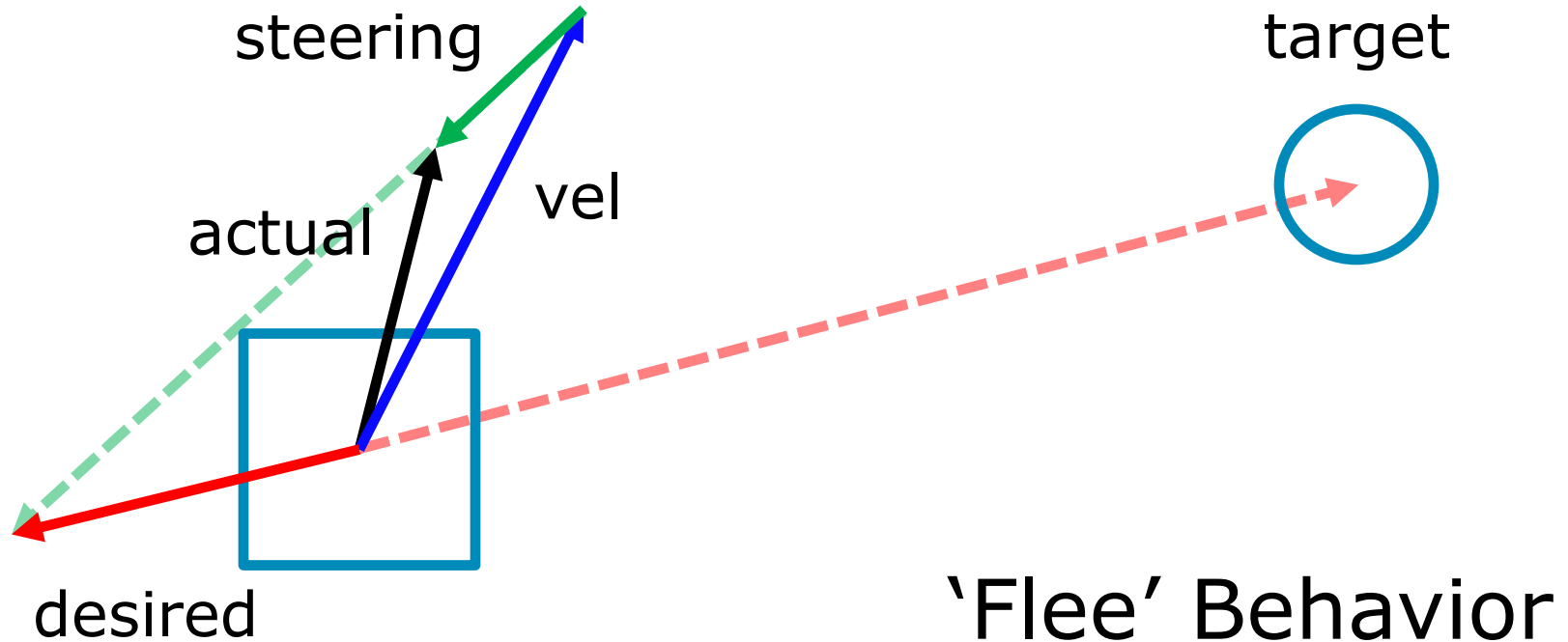


'Seek' Behavior

Types of Steering

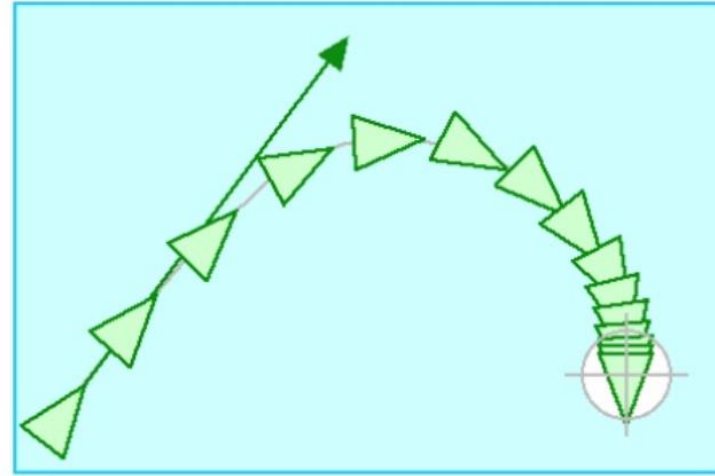


Types of Steering



Arrive Behavior

- One issue with steering is close to the target the entity will oscillate
- Must define an 'arrive' behavior
- Simplest solution is to scale speed linearly as it approaches target within some dist
- No 'best' values / algorithm, just try it!



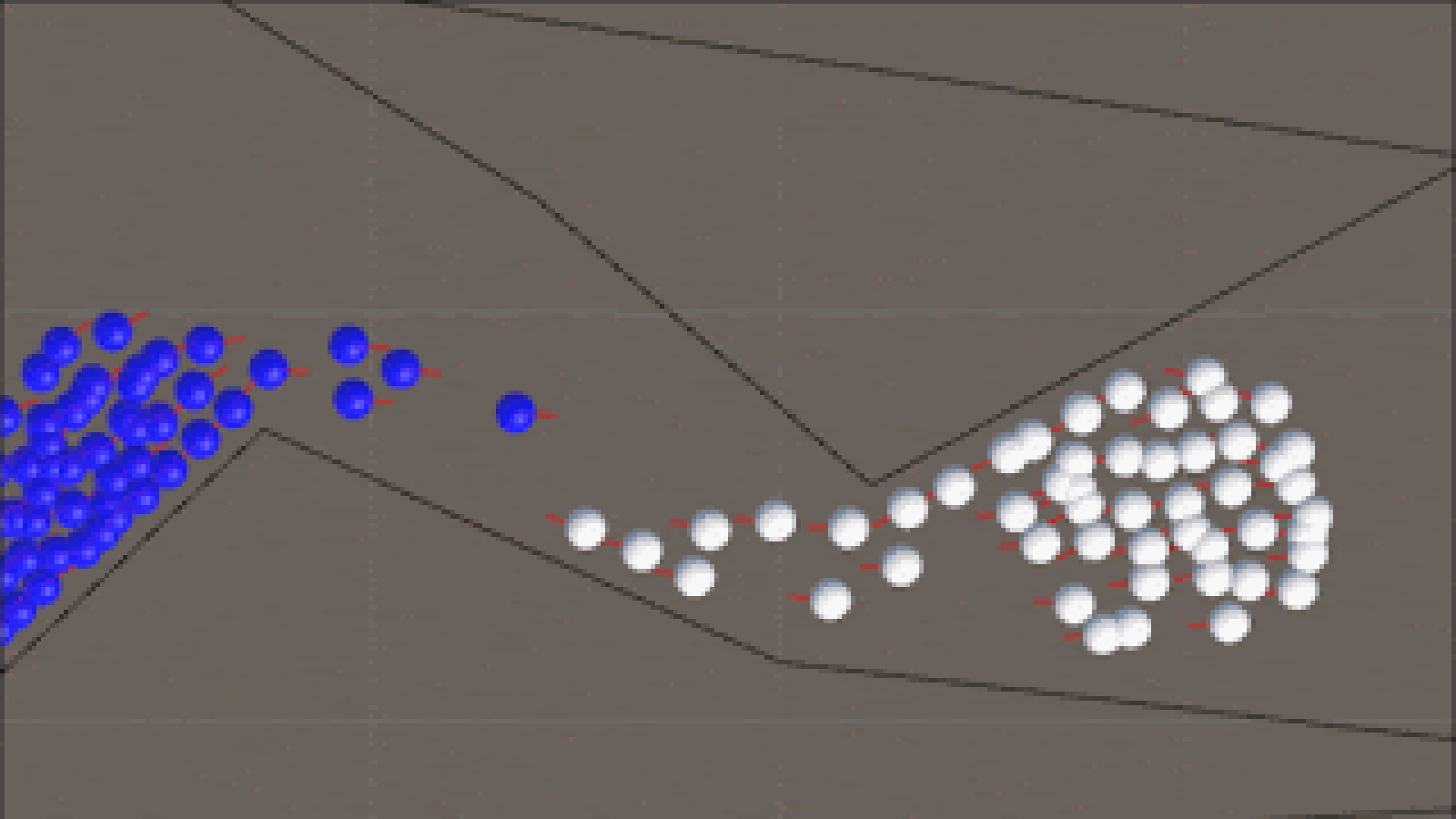
Seek

Seek
Steering



Avoidance Steering





Steering Demo

<http://opensteer.sourceforge.net>